Projet fil rouge 2026 — Bachelor 3 Cybersécurité

InfraReady

Rapport final de projet

Standardiser, automatiser et superviser une infrastructure Linux / Windows

Laboratoire Proxmox de trois serveurs — Active Directory, Ansible, Prometheus, Grafana et intégration continue GitHub Actions

Équipe	MIR Mahan Jolan Allen (chef de projet) Kenzi Bendjelloul (automatisation Ansible)
Formation	Bachelor 3 Cybersécurité — Administration systèmes Linux / Windows
Période	19 au 23 octobre 2026
Dépôt de remise	`https://github.com/jolanallen/InfraReady`
Version	1.0 — 23/10/2026

Toutes les valeurs, captures et sorties citées proviennent du laboratoire réel et sont conservées, horodatées, dans `docs/preuves/` du dépôt (preuves numérotées **P-xx**). Aucun mot de passe ni secret n'apparaît dans ce document.

Résumé

L'entreprise fictive *InfraReady* souhaite standardiser la configuration de ses serveurs, automatiser leur administration et les superviser. Le projet consistait à concevoir, déployer et documenter un laboratoire représentatif : trois machines virtuelles sur un hyperviseur Proxmox VE, dans un réseau isolé `192.168.50.0/24` — un contrôleur de domaine Windows Server 2025 (**WIN-DC01**, Active Directory et DNS), un serveur Ubuntu 24.04 hébergeant un service web (**LINUX01**) et un serveur d'exploitation Ubuntu 24.04 portant Ansible, Prometheus et Grafana (**OPS01**).

Le travail a été mené en huit phases (A à H) puis complété par deux séances de travaux pratiques. Les installations sont automatiques, la configuration des serveurs est décrite par des playbooks Ansible rejoués deux fois pour prouver leur idempotence, les secrets sont chiffrés par Ansible Vault, et les métriques des trois serveurs sont collectées par Prometheus et présentées dans un tableau de bord Grafana. Un dépôt GitHub versionne l'ensemble ; chaque modification passe par une branche et une demande de fusion validée par un pipeline GitHub Actions (lint, syntaxe, règles Prometheus, recherche de secrets).

La séance 6, consacrée à la maintenance opérationnelle et à la supervision, a ajouté un contrôle de santé automatisé en lecture seule, une tâche planifiée qui teste le site web chaque minute, deux alertes vérifiées par des pannes contrôlées, un tableau de bord d'exploitation, des quotas disque et le diagnostic d'un incident réel : l'arrêt brutal du contrôleur de domaine à chaque arrêt de l'hyperviseur.

Résultats clés

— 3 serveurs opérationnels, domaine `infraready.test` fonctionnel, isolation réseau vérifiée ;

— 8 playbooks Ansible ; ceux de configuration sont idempotents (second passage `changed=0`) ;

— 4/4 cibles Prometheus collectées, 7 règles d'alerte, dashboard Grafana de 9 panneaux ;

— 16 demandes de fusion (PR), pipeline CI vert, gitleaks sans alerte sur tout l'historique ;

— plus de 100 preuves horodatées ; tous les échecs rencontrés sont documentés avec leur cause et leur correction.

Points ouverts à la remise

La correction de l'arrêt non propre du contrôleur de domaine (incident INC-2026-10-23-01) est diagnostiquée, reproduite et préparée mais n'est pas appliquée : elle modifie le contrôleur de domaine partagé et attend la validation de l'équipe. Les bilans individuels restent à rédiger par chaque membre. L'emploi de Proxmox VE à la place de VMware Workstation doit être validé par le formateur.

Mots-clés : Proxmox VE, Windows Server 2025, Active Directory, Ubuntu Server, Ansible, idempotence, Ansible Vault, Prometheus, Grafana, node_exporter, windows_exporter, GitHub Actions, maintenance, supervision, diagnostic d'incident.

Table des matières

Résumé i

Table des figures iv

Liste des tableaux vi

1 Introduction 1
1.1 Contexte . . . 1
1.2 Problématique . . . 1
1.3 Objectifs . . . 2
1.4 Périmètre et contraintes . . . 2
1.5 Démarche et organisation du document . . . 2

2 Organisation du projet 4
2.1 Équipe et répartition des rôles . . . 4
2.2 Méthode de travail . . . 4
2.3 Planning réel . . . 5
2.4 Outils et versions . . . 7
2.5 Conventions . . . 7

3 Architecture 8
3.1 Vue d'ensemble . . . 8
3.2 Inventaire des serveurs . . . 10
3.3 Flux autorisés et filtrage . . . 11
3.4 Choix techniques et écarts au sujet . . . 11

4 Réalisation — phases A à C 12
4.1 Phase A — Installation et réseau . . . 12
4.1.1 Réalisation . . . 12
4.1.2 Résultats . . . 13
4.1.3 Difficultés et décisions . . . 14
4.2 Phase B — Administration Windows . . . 14
4.2.1 Réalisation . . . 14
4.2.2 Résultats . . . 15
4.2.3 Difficultés et décisions . . . 15
4.3 Phase C — Administration Linux . . . 16
4.3.1 Réalisation . . . 16
4.3.2 Résultats . . . 17

5 Réalisation — phases D à H 18
5.1 Phase D — Automatisation avec Ansible . . . 18
5.1.1 Réalisation . . . 18
5.1.2 Résultats . . . 20

5.1.3 Difficultés et décisions . . . 20
5.2 Phase E — Maintenance et mises à jour . . . 20
5.3 Phase F — Supervision avec Prometheus et Grafana . . . 22
5.3.1 Réalisation . . . 22
5.3.2 Résultats . . . 23
5.4 Phase G — GitHub et intégration continue . . . 25
5.5 Phase H — Diagnostic d'incident . . . 25

6 Séance 4 — Stockage, processus et scénarios de panne 28
6.1 Correspondance avec le support . . . 28
6.2 Scripts de vérification . . . 28
6.3 Scénarios de panne contrôlée . . . 29
6.4 Enseignements . . . 29

7 Séance 6 — Maintenance opérationnelle et supervision 31
7.1 Déroulé et bilan . . . 31
7.2 UC-01 — État de santé vérifiable . . . 33
7.3 UC-02 — Métriques Linux et Windows collectables . . . 33
7.4 UC-03 — Tableau de bord d'exploitation . . . 35
7.5 UC-04 et UC-C4 — Tâche planifiée et alertes testées . . . 35
7.6 UC-05 — Incident : arrêt brutal du contrôleur de domaine . . . 39
7.7 UC-06 — Intégration aux livrables . . . 42
7.8 UC-C1 et UC-C2 — Maintenance encadrée par un contrôle de santé . . . 42
7.9 UC-C3 — Observation ponctuelle et suivi dans le temps . . . 45
7.10 UC-A3 (défi) — Quotas sur le disque de test . . . 45

8 Sécurité, limites d'intervention et recette 47
8.1 Mesures de sécurité appliquées . . . 47
8.2 Limites connues . . . 48
8.3 Recette finale . . . 48
8.4 Checklist de fin de séance 6 (support, section 6.1) . . . 49

9 Bilan 50
9.1 Principales difficultés . . . 50
9.2 Écarts au sujet . . . 50
9.3 Fonctionnalités inachevées . . . 51
9.4 Perspectives . . . 51
9.5 Bilan de l'équipe . . . 51
Conclusion . . . 52

A Matrice des preuves 53

B Arborescence du dépôt 58

C Journal des changements 59

D Glossaire 62

E Références 64

Table des figures

2.1 Cycle appliqué à chaque changement, du laboratoire au dépôt . . . 5
2.2 Planning réel du projet (19, 20 et 23 octobre 2026) . . . 6

3.1 Architecture réellement déployée : hôte Proxmox, pont `vmbr1`, trois serveurs et flux principaux . . . 9

4.1 WIN-DC01 installé automatiquement : Server Manager (**P-03**) . . . 13

5.1 Origine, ports et cheminement des métriques (phase F) . . . 22
5.2 Cibles Prometheus : 4/4 `up` (**P-51**) . . . 23
5.3 Dashboard Grafana (version 1) pendant la démonstration : LINUX01 hors ligne et alerte active (**P-55**) . . . 24
5.4 Chronologie de l'incident de la phase H . . . 26

7.1 Déroulé réel de la séance 6 (en rouge : blocage et tests de panne contrôlés) . . . 32
7.2 Snapshots des trois VM, dont `s09_avant_seance6` (rendu de la sortie de l'API, **P-80c**) 33
7.3 Architecture de supervision après la séance 6 (en bleu : éléments ajoutés) . . . 34
7.4 Prometheus : 4/4 cibles `up` après correction (**P-81**) . . . 34
7.5 Dashboard Grafana v3 : le laboratoire était éteint avant 9 h, d'où les zones vides plutôt que des zéros (**P-85**) . . . 35
7.6 Chaîne de détection pendant le test 1 (nginx arrêté) . . . 36
7.7 Cycle de vie des deux alertes pendant les tests . . . 37
7.8 Dashboard figé à 09:29:40 : contrôle du site en ÉCHEC et alerte `SiteWebIndisponible` active, alors que LINUX01 reste EN LIGNE (**P-84**) . . . 38
7.9 Prometheus : `infraready_controle_succes` tombe à 0 pendant l'arrêt de nginx (axe horaire en UTC, soit l'heure de Paris moins 2 h) . . . 39
7.10 Timer, journal du contrôle et métriques publiées sur LINUX01 (rendu de la sortie, **P-83**) . . . 39
7.11 Événements 41 et 6008 de WIN-DC01 (rendu de la sortie, **P-80**) . . . 39
7.12 Démarche de diagnostic de l'incident INC-2026-10-23-01 . . . 40
7.13 Ce qui se passe à l'arrêt de l'hôte . . . 41
7.14 Journal des tâches Proxmox : la VM 100 tuée après 182–183 s, puis la reproduction contrôlée qui échoue sur délai (rendu de la sortie, **P-86**) . . . 41
7.15 Du laboratoire au dépôt : branches, demandes de fusion, CI et fusion . . . 42
7.16 Demandes de fusion de la séance 6, fusionnées avec des contrôles CI 8/8 (**P-64**) . . . 42
7.17 Structure du playbook `controle_sante.yml` . . . 43
7.18 Contrôle de santé des trois serveurs : 23 OK, 3 ATTENTION, 1 NON APPLICABLE, 0 ECHEC (rendu de la sortie, **P-80b** / **P-82**) . . . 43
7.19 Processus de maintenance suivi, avec ses points d'arrêt . . . 44
7.20 Maintenance de LINUX01 : 3 paquets installés, 5 reportés par le phasage d'Ubuntu, aucun écart de statut avant / après (rendu de la sortie, **P-87**) . . . 44
7.21 Périodes couvertes par l'historique Prometheus . . . 45

7.22 Évolution du stockage du 19 au 23/10 : trois îlots de données ; on y voit aussi `C:` de WIN-DC01 passer de 24 % à 43 % puis 37 % pendant Windows Update (**P-88**) . . 45
7.23 Comportement des quotas observé 46
7.24 Test réel : alice acceptée à 220 Mio avec délai de grâce, puis bloquée à 250 Mio (rendu de la sortie, **P-89**) 46

Liste des tableaux

1.1 Objectifs du projet et critère de réussite retenu . . . 2

2.1 Répartition des rôles dans l'équipe . . . 4
2.2 Outils utilisés et versions réellement déployées . . . 7

3.1 Serveurs du laboratoire et ressources réelles . . . 10
3.2 Paramètres réseau et domaine . . . 10
3.3 Matrice des flux autorisés . . . 11
3.4 Choix techniques et justification . . . 11

4.1 Phase A — vérifications et résultats . . . 13
4.2 Phase A — difficultés rencontrées . . . 14
4.3 Groupes, membres et droits attribués (principe du moindre privilège) . . . 15
4.4 Phase B — vérifications et résultats . . . 15
4.5 Phase B — difficultés rencontrées . . . 15
4.6 Comptes, groupes et permissions sur LINUX01 . . . 16
4.7 Phase C — vérifications et résultats . . . 17

5.1 Playbooks du dépôt . . . 19
5.2 Phase D — vérifications et résultats . . . 20
5.3 Phase D — difficultés rencontrées . . . 20
5.4 Étapes de la maintenance outillée . . . 21
5.5 Phase E — vérifications et résultats . . . 21
5.6 Règles d'alerte (5 en phase F, 7 après la séance 6) . . . 23
5.7 Phase F — vérifications et résultats . . . 24
5.8 Jobs du pipeline GitHub Actions . . . 25
5.9 Phase H — démarche et résultats . . . 27

6.1 Étapes du support et réalisation . . . 28
6.2 Codes de sortie des scripts de vérification . . . 29
6.3 Six scénarios de panne contrôlée . . . 29

7.1 Bilan de la séance 6 par use case . . . 32
7.2 Écarts relevés par l'état initial et suite donnée . . . 33
7.3 Les deux alertes ajoutées . . . 36

8.1 Mesures de sécurité et de maîtrise des risques . . . 47
8.2 Recette finale par phase . . . 48
8.3 Checklist de fin de séance 6 . . . 49

9.1 Familles de difficultés et enseignements . . . 50
9.2 Points ouverts et action nécessaire . . . 51

A.1 Matrice des preuves (`docs/preuves/`) . . . 53

C.1 Journal des changements sur le laboratoire 59

Chapitre 1

Introduction

1.1 Contexte

InfraReady est une PME de services fictive qui prépare la mise en exploitation d'une petite infrastructure hybride. Son équipe informatique administre aujourd'hui ses serveurs à la main : les configurations divergent d'une machine à l'autre, les opérations ne sont pas reproductibles et les indisponibilités sont découvertes par les utilisateurs. Elle souhaite :

— **standardiser** la configuration de ses serveurs Linux et Windows ;

— **automatiser** les tâches d'administration et de maintenance ;

— **superviser** l'état des serveurs pour détecter les anomalies avant les utilisateurs ;

— **tracer** chaque changement dans un dépôt Git validé automatiquement, afin qu'un autre administrateur puisse comprendre et réutiliser les configurations.

1.2 Problématique

Comment construire, avec des outils libres ou d'évaluation, une infrastructure de laboratoire dont l'état est *décrit par un dépôt* plutôt que par les manipulations de ses administrateurs, dont chaque changement est *vérifiable* et dont le bon fonctionnement est *observable* en continu — tout en restant isolée du réseau personnel et sans exposer aucun secret ?

1.3 Objectifs

Table 1.1 : Objectifs du projet et critère de réussite retenu

Objectif	Critère de réussite	Où le vérifier
Laboratoire isolé	3 VM joignables entre elles et vers Internet, réseau domestique bloqué	Phase A, **P-02**
Annuaire et DNS	Domaine `infraready.test` fonctionnel, `dcdiag` réussi	Phase B, **P-12b**
Administration Linux	Comptes, droits, service web, disque de test, journaux vérifiés	Phase C, **P-20** à **P-24**
Automatisation	Playbooks Linux et Windows, second passage `changed=0`	Phase D, **P-32**, **P-34**
Maintenance	Procédure outillée avec pré- et post-contrôles, rapports	Phase E, **P-40** à **P-43**
Supervision	Métriques Linux et Windows, dashboard, alerte démontrée	Phase F, **P-50** à **P-56**
Versionnement et CI	Pipeline GitHub Actions vert, aucun secret	Phase G, **P-60**, **P-64**
Diagnostic	Incident reproduit, cause confirmée, retour à la normale prouvé	Phase H, **P-70** à **P-73**

1.4 Périmètre et contraintes

— **Laboratoire uniquement** : trois VM dédiées (VMID 100 à 102) ; les autres machines de l'hôte ne sont jamais modifiées. Aucune donnée réelle : comptes, groupes et données sont fictifs.

— **Isolation** : le réseau du laboratoire accède à Internet (mises à jour) mais ne peut ouvrir aucune connexion vers le réseau domestique.

— **Secrets** : aucun mot de passe en clair dans le dépôt ni dans ce document ; les secrets vivent dans des fichiers Ansible Vault chiffrés.

— **Réversibilité** : snapshot Proxmox avant chaque changement important ; toute panne volontaire est contrôlée et réversible.

1.5 Démarche et organisation du document

Le projet suit le découpage du sujet en huit phases (A à H), complété par les travaux pratiques des séances 4 (stockage, processus, scénarios de panne) et 6 (maintenance opérationnelle et supervision). Chaque phase suit le même cycle (chapitre 2) : planifier, protéger par un snapshot, réaliser de préférence avec Ansible, rejouer pour vérifier l'idempotence, capturer une preuve horodatée, documenter, puis livrer par une demande de fusion validée par la CI.

Le document est organisé ainsi :

— chapitre 2 — organisation de l'équipe, méthode, planning et outils ;

— chapitre 3 — architecture cible et réellement déployée ;

— chapitres 4 et 5 — réalisation des phases A à H : objectif, réalisation, résultats mesurés, difficultés et retour arrière ;

— chapitres 6 et 7 — travaux pratiques des séances 4 et 6 ;

— chapitre 8 — sécurité, limites d'intervention et recette finale ;

— chapitre 9 — difficultés, écarts, limites, bilan et perspectives, conclusion ;

— annexes — matrice des preuves, arborescence du dépôt, journal des changements, glossaire.

Comment lire les renvois aux preuves

Chaque résultat cite une preuve **P-xx** : un fichier de `docs/preuves/` contenant la sortie horodatée des commandes exécutées (en-tête : date, hôte, commande) ou une capture d'écran. La numérotation suit les phases : A = P-0x, B = P-1x, C = P-2x, D = P-3x, E = P-4x, F = P-5x, G = P-6x, H = P-7x, séance 6 = P-8x. Un suffixe « a » (P-10a, P-39a...) désigne un premier essai en échec, conservé et expliqué. Les images nommées `*-rendu-*.png` sont des rendus visuels de sorties de commandes réelles ; leur texte intégral est dans la preuve `.txt` correspondante.

Chapitre 2

Organisation du projet

2.1 Équipe et répartition des rôles

Table 2.1 : Répartition des rôles dans l'équipe

Membre	Rôle et responsabilités
Jolan Allen	Chef de projet : planification, coordination, suivi, dépôt de remise `jolanallen/InfraReady`, relecture et fusion des demandes de fusion ; script d'idempotence, contrôles du vault, filtre d'hôte du dashboard (PR #1 à #5)
Kenzi Bendjelloul	Automatisation : partie Ansible (phase D)
MIR Mahan	Réalisation du laboratoire et des phases, preuves, documentation, travaux des séances 4 et 6

Les phases A à C et E à H ont été réalisées conjointement par MIR Mahan et Jolan Allen ; les séances de travaux pratiques par les trois membres.

2.2 Méthode de travail

La figure 2.1 résume le cycle appliqué à chaque changement. Trois principes le guident :

Le dépôt décrit le laboratoire. Une modification faite à la main sur une VM est rejouée en Ansible ; on ne corrige pas un fichier en place sur un serveur.

Une preuve par résultat. Les sorties de commandes sont capturées automatiquement avec l'heure, l'hôte et la commande exécutée. Les échecs sont conservés.

Un sujet, une branche, une demande de fusion. Depuis le 20/10, aucune modification n'est poussée directement sur `main` ; chaque demande de fusion est validée par la CI puis relue et fusionnée par le chef de projet.

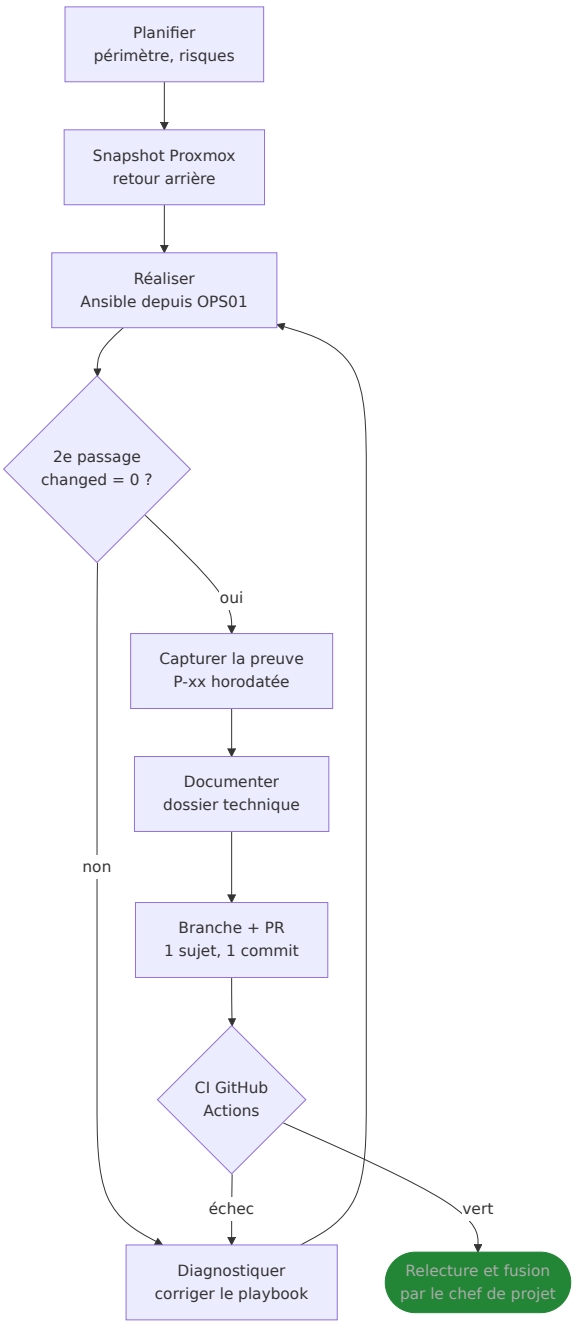


FIGURE 2.1 : Cycle appliqué à chaque changement, du laboratoire au dépôt

2.3 Planning réel

Le projet s'est déroulé sur trois journées de travail effectif. La figure 2.2 montre l'enchaînement réel, reconstitué à partir du journal des changements (annexe C).

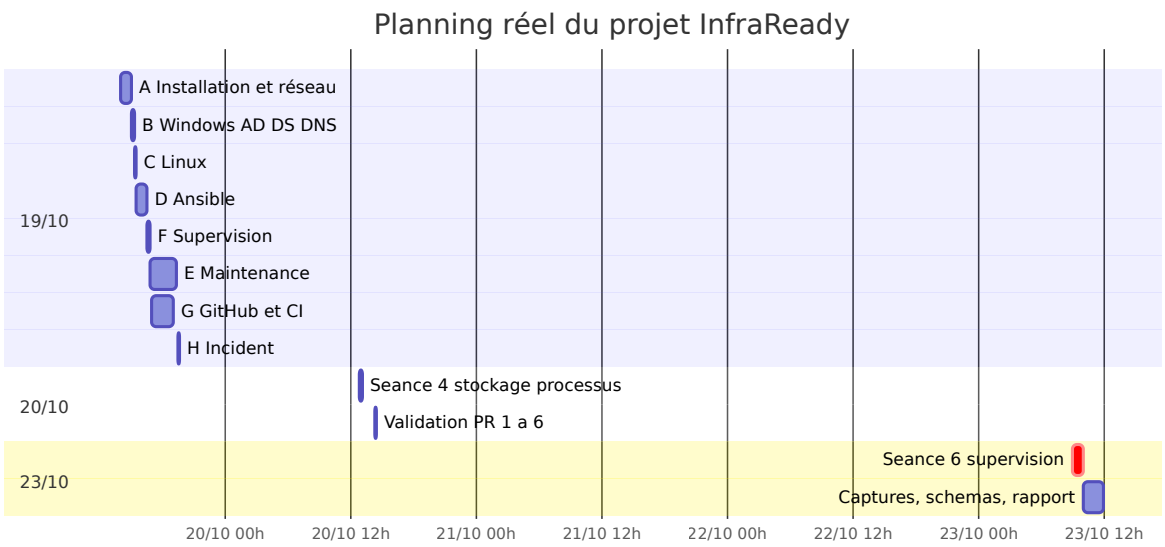


FIGURE 2.2 : Planning réel du projet (19, 20 et 23 octobre 2026)

2.4 Outils et versions

TABLE 2.2 : Outils utilisés et versions réellement déployées

Domaine	Outil (version)	Usage
Virtualisation	Proxmox VE 9.1.1	Hôte `corsair1`, 3 VM, snapshots, API
Systèmes	Windows Server 2025 Standard Evaluation ; Ubuntu Server 24.04.5 LTS	WIN-DC01 ; LINUX01 et OPS01
Automatisation	ansible-core 2.16.3, collections `ansible.windows` 2.2.0, `ansible.posix` 1.5.4, `community.general` 8.3.0, `community.windows` 2.1.0, `microsoft.ad` 1.4.1	Playbooks depuis OPS01 (SSH et WinRM)
Secrets	Ansible Vault (AES256)	Mots de passe Administrateur, DSRM, comptes AD, Grafana
Supervision	Prometheus 2.45.3, node_exporter 1.7.0, windows_exporter 0.31.8, Grafana 13.2.4	Collecte, alertes, tableau de bord
Service	nginx 1.24	Site web de LINUX01
Versionnement	Git, GitHub, GitHub Actions	Dépôt, demandes de fusion, pipeline de validation
Qualité	yamllint, ansible-lint 24 (profil *production*), promtool, jq, gitleaks 8.24	Contrôles locaux et en CI
Documentation	Obsidian (Markdown, Mermaid), LuaLaTeX	Dossier technique, ce rapport

2.5 Conventions

— **Preuves** : `docs/preuves/P-<phase><n>-<description>.txt|png`, en-tête avec date, hôte et commandes ; premiers essais en échec suffixés « a ».

— **Snapshots** : `s<nn>_<moment>` (par exemple `s04_avant_maj_windows`, `s09_avant_seance6`).

— **Commits et demandes de fusion** : en français, sujet au présent, corps qui explique le constat et la correction ; branches `fix/`, `feat/`, `docs/`, `ci/`.

— **Secrets** : recherche des vraies valeurs dans les fichiers modifiés avant chaque commit, puis gitleaks sur tout l'historique dans la CI.

Chapitre 3

Architecture

3.1 Vue d'ensemble

Le laboratoire est hébergé sur un hôte Proxmox VE (`corsair1`, 192.168.0.7) du réseau domestique. Un pont dédié `vmbr1`, sans carte physique, porte le réseau du laboratoire `192.168.50.0/24` ; l'hôte joue le rôle de passerelle (`192.168.50.1`) avec une traduction d'adresses vers Internet et un filtrage qui interdit toute nouvelle connexion vers le réseau domestique (figure 3.1).

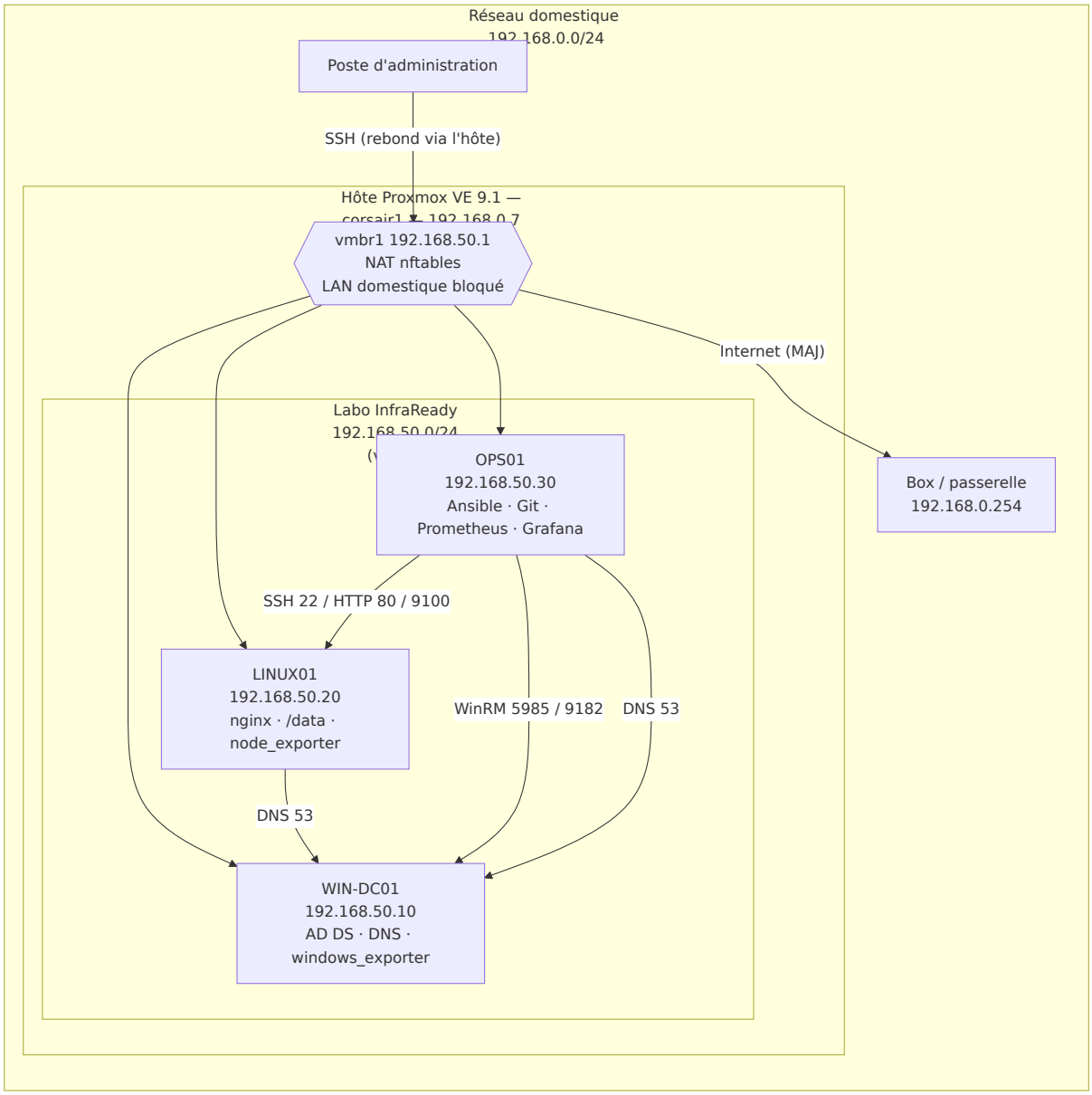


FIGURE 3.1 : Architecture réellement déployée : hôte Proxmox, pont `vmbr1`, trois serveurs et flux principaux

3.2 Inventaire des serveurs

TABLE 3.1 : Serveurs du laboratoire et ressources réelles

Machine	VMID	Système	Rôle	Ressources	Adresse
WIN-DC01	100	Windows Server 2025 Standard Evaluation (Desktop Experience)	AD DS, DNS	4 vCPU, 6 Go, 50 Go ; UEFI + TPM 2.0 ; carte e1000e	192.168.50.10
LINUX01	101	Ubuntu Server 24.04.5 LTS	Service web nginx, stockage de test	4 vCPU, 4 Go, 30 Go + disque de test 10 Go (`/data`)	192.168.50.20
OPS01	102	Ubuntu Server 24.04.5 LTS	Ansible, Git, Prometheus, Grafana	8 vCPU, 6 Go, 50 Go	192.168.50.30

TABLE 3.2 : Paramètres réseau et domaine

Paramètre	Valeur
Sous-réseau du laboratoire	`192.168.50.0/24`, passerelle `192.168.50.1` (hôte Proxmox)
Domaine Active Directory	`infraready.test` (NetBIOS `INFRAREADY`), niveau fonctionnel Windows 2025
DNS	WIN-DC01 (`192.168.50.10`) pour tout le laboratoire ; redirecteurs 1.1.1.1 et 9.9.9.9
Heure	WIN-DC01 synchronisé sur `fr.pool.ntp.org` ; VM Linux synchronisées par NTP
Stockage	Disques sur le pool ZFS `pool-ssd` de l'hôte
Hôte	16 cœurs, environ 46 Go de mémoire : aucune réduction de ressources nécessaire

3.3 Flux autorisés et filtrage

TABLE 3.3 : Matrice des flux autorisés

Source	Destination	Port	Justification	Contrôle
Laboratoire	Internet	tous (sortant)	Mises à jour, téléchargements	NAT nftables sur l'hôte
Laboratoire	Réseau domestique	—	**Interdit** (isolation)	nftables : `ct state new drop`
Laboratoire	LINUX01	22/tcp	Administration SSH par clé	UFW, mot de passe désactivé
Laboratoire	LINUX01	80/tcp	Service web	UFW
OPS01	LINUX01	9100/tcp	node_exporter	UFW (OPS01 seul)
Laboratoire	WIN-DC01	5985/tcp	WinRM (Ansible)	Pare-feu Windows limité au laboratoire
OPS01	WIN-DC01	9182/tcp	windows_exporter	Pare-feu Windows (OPS01 seul)
Laboratoire	WIN-DC01	53, 88, 389, 445…	DNS, Kerberos, LDAP, SMB	Règles AD par défaut
Laboratoire	OPS01	22, 3000, 9090/tcp	SSH, Grafana, Prometheus	UFW

3.4 Choix techniques et écarts au sujet

TABLE 3.4 : Choix techniques et justification

Choix	Justification
Proxmox VE au lieu de VMware Workstation	Snapshots et API scriptables, ajout de disque à chaud, installations pilotables en ligne de commande, ressources suffisantes. **Écart à faire valider par le formateur.**
Installations automatiques	Ubuntu autoinstall (cloud-init) et Windows `autounattend.xml` : aucune saisie manuelle, installations reproductibles (modèles dans `lab/`).
nftables plutôt qu'iptables	Proxmox VE 9 n'installe pas iptables ; table dédiée `infraready_nat` chargée avec le pont.
OPS01 comme nœud de contrôle	Ansible, Prometheus et Grafana sur une même VM d'exploitation ; les serveurs administrés n'ont pas d'outil d'administration en plus.
WinRM HTTP + NTLM	Messages chiffrés par NTLM sans certificat TLS ; acceptable dans un laboratoire isolé (limite documentée).
Ubuntu 24.04.5	Dernière révision de la 24.04 LTS demandée par le sujet.

Chapitre 4

Réalisation — phases A à C

Chaque phase est présentée de la même façon : objectif, réalisation, résultats mesurés (avec la preuve correspondante), difficultés rencontrées et moyen de retour arrière.

4.1 Phase A — Installation et réseau

Objectif

Mettre en service les machines virtuelles, leurs noms, leurs adresses et la connectivité du laboratoire.

Responsables : MIR Mahan, Jolan Allen · **Dates :** 19/10/2026 · **Statut :** Fait

4.1.1 Réalisation

Réseau. Après sauvegarde de `/etc/network/interfaces`, le pont `vmbr1` (`lab/proxmox/vmbr1.interfaces`) et une table nftables dédiée (`lab/proxmox/infraready-nat.nft`) ont été ajoutés sur l'hôte : traduction d'adresses (*masquerade*) du laboratoire vers Internet, et rejet de toute nouvelle connexion du laboratoire vers `192.168.0.0/24`.

Installations automatiques. Aucune saisie manuelle n'a été nécessaire :

— **LINUX01 et OPS01** : ISO Ubuntu 24.04.5 (empreinte SHA-256 vérifiée) et second lecteur « CIDATA » portant un fichier *autoinstall* : nom, adresse fixe, LVM, clavier français, compte d'administration avec mot de passe haché, SSH par clé uniquement, agent invité QEMU, mises à jour de sécurité ;

— **WIN-DC01** : ISO Windows Server 2025 et fichier `autounattend.xml` : image Standard avec interface graphique, partitionnement UEFI/GPT, nom, adresse fixe, fuseau horaire, WinRM et ping autorisés depuis le laboratoire uniquement.

Les fichiers de réponse contenant de vrais mots de passe ont été supprimés du stockage Proxmox après usage ; le dépôt ne contient que des modèles à variables (`lab/*.tpl`).

DNS. Une fois le domaine créé (phase B), les VM Linux interrogent le contrôleur de domaine (`nameservers 192.168.50.10`, domaine de recherche `infraready.test`).

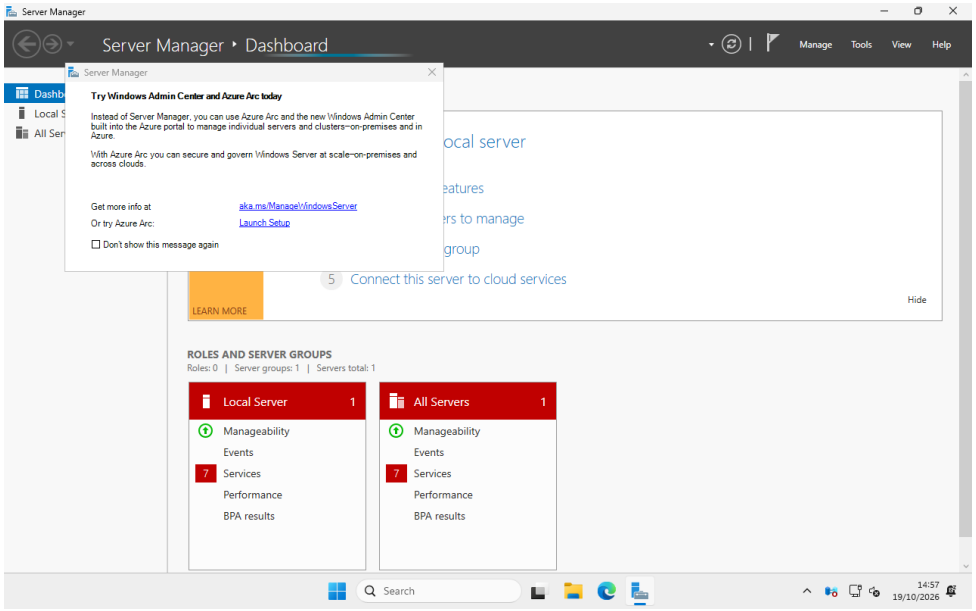


FIGURE 4.1 : WIN-DC01 installé automatiquement : Server Manager (**P-03**)

4.1.2 Résultats

TABLE 4.1 : Phase A — vérifications et résultats

Vérification	Résultat observé	Preuve
Trois VM	3 VM `running`, snapshot `s01_installation_ok` ; Windows installé en environ 10 min, Ubuntu en 6 min	**P-01**, **P-03**
Adressage	.10 / .20 / .30, passerelle .1, toutes sur `vmbr1`	**P-02**, **P-02b**, **P-04c**
Connectivité	Ping entre les 3 VM et vers la passerelle ; Internet joignable (HTTP 200)	**P-02**, **P-02b**
Isolation	`192.168.0.254` bloqué depuis le laboratoire mais joignable depuis l'hôte : le blocage vient bien de la règle nftables	**P-02**
DNS	Enregistrements A, PTR et SRV LDAP résolus depuis les VM Linux et le DC	**P-04**, **P-04b**, **P-04c**

4.1.3 Difficultés et décisions

Table 4.2 : Phase A — difficultés rencontrées

Problème	Cause	Correction et résultat
ISO Ubuntu 26.04 attachée aux VM Linux	Mauvaise ISO à la création	ISO 24.04.5 téléchargée, empreinte vérifiée, remplacée
Carte réseau e1000 sur WIN-DC01	Pilote non garanti sous Windows Server 2025	Passage en e1000e (même adresse MAC)
L'installeur Ubuntu demande une confirmation	Mot-clé `autoinstall` absent de la ligne de commande	Réponse envoyée par `qm sendkey`
Pas d'iptables sur l'hôte	Proxmox VE 9 repose sur nftables	Règles écrites en nftables
Enregistrement PTR du DC absent	Zone inverse créée après la promotion	Enregistrement dynamique déclenché par `windows_base.yml` (phase D)
Mot de passe sudo de l'hôte visible dans l'en-tête d'une preuve	Le script de capture recopiait la commande	Preuve nettoyée, script corrigé, recherche des secrets avant chaque commit

Retour arrière : snapshot `s01_installation_ok` des 3 VM, sauvegarde `interfaces.bak-infraready` sur l'hôte.

4.2 Phase B — Administration Windows

Objectif

Rendre l'annuaire et le DNS opérationnels, documenter comptes, groupes, droits et journaux.

Responsables : MIR Mahan, Jolan Allen · **Dates :** 19/10/2026 · **Statut :** Fait

4.2.1 Réalisation

Après le snapshot `s02_avant_adds`, toutes les actions ont été lancées depuis OPS01 par WinRM :

— **Forêt AD** (`windows_adds.yml`) : forêt `infraready.test`, NetBIOS `INFRAREADY`, niveau fonctionnel Windows 2025, DNS intégré à AD avec redirecteurs, NTP externe (indispensable à Kerberos), mot de passe DSRM lu dans le vault chiffré ;

— **Objets de l'annuaire** (`scripts/ad-objets.ps1`, rejouable) : unités d'organisation, deux groupes globaux de sécurité et quatre comptes, avec changement de mot de passe obligatoire à la première connexion.

TABLE 4.3 : Groupes, membres et droits attribués (principe du moindre privilège)

Groupe	Membres	Droits sur le partage Comptabilité
`GG_Comptabilite`	l.petit, t.moreau	Modification (partage SMB « Change », NTFS « Modify »)
`GG_Support_IT`	j.martin, s.bernard	Lecture seule, pour dépanner
Autres utilisateurs	—	Aucun accès : `BUILTIN\Users` retiré, héritage désactivé, partage masqué

4.2.2 Résultats

TABLE 4.4 : Phase B — vérifications et résultats

Vérification	Résultat observé	Preuve
Domaine	`infraready.test`, DC `WIN-DC01.infraready.test`, catalogue global ; `dcdiag` : 10 tests fonctionnels réussis	**P-10**, **P-12**, **P-12b**
Comptes et groupes	2 groupes, 4 comptes, appartenances conformes	**P-11**
DNS interne	A et SRV LDAP résolus, résolution externe par les redirecteurs, zones en mise à jour sécurisée	**P-12**, **P-04c**
Journaux	Redémarrage de promotion (1074), démarrage d'AD DS (1000), zones DNS chargées, création des comptes et groupes tracée (4720, 4727, 4728)	**P-13**

4.2.3 Difficultés et décisions

TABLE 4.5 : Phase B — difficultés rencontrées

Problème	Cause	Correction et résultat
1er lancement de `windows_adds.yml` en échec	Paramètre `reboot_timeout` absent de `microsoft.ad` 1.4.1	Paramètre retiré ; forêt créée au 2e lancement (**P-10a** → **P-10**)
DC synchronisé sur sa propre horloge	Configuration par défaut	NTP externe : strate 2 (**P-12b**)
Événements horodatés dans le futur	Hypothèse : saut d'horloge pendant la promotion, avant NTP	Heure vérifiée identique à OPS01 ; plus de dérive
« Le DC utilise son propre DNS » signale un changement à chaque passage	`win_dns_client` retire `::1` que Windows remet sur un DC	Liste `127.0.0.1` et `::1` : 2e passage `changed=0` (**P-15**)

Retour arrière : snapshots `s02_avant_adds` et `s03_avant_ansible` de WIN-DC01.

4.3 Phase C — Administration Linux

Objectif

Gérer identités, permissions, service, stockage, mises à jour et journaux sur Ubuntu.

Responsables : MIR Mahan, Jolan Allen · **Dates :** 19/10/2026 · **Statut :** Fait

4.3.1 Réalisation

Réalisée d'abord à la main sur LINUX01 (commandes conservées dans les preuves), puis reprise par Ansible en phase D.

TABLE 4.6 : Comptes, groupes et permissions sur LINUX01

Élément	Configuration	Effet
alice, bruno	groupe `webadmins`	Mettent à jour le site web
chloe	groupes `auditeurs`, `systemd-journal`	Lit les journaux et rapports, sans droit d'écriture
ansible	sudo sans mot de passe, clé SSH	Compte de service de l'automatisation
`/var/www/infraready`	`root:webadmins`, mode 2775 (setgid)	Les webadmins écrivent, les fichiers héritent du groupe
`/data/rapports`	`root:auditeurs`, mode 2750	Les auditeurs lisent, les autres n'ont aucun accès

— **Service** : nginx 1.24 avec un site dédié et des journaux séparés ; `nginx -t` avant chaque rechargement.

— **Stockage** : disque de 10 Go ajouté à chaud et identifié par son **numéro de série** (`INFRA-DATA01`), et non par un nom de périphérique supposé. Le formatage n'a lieu que si cinq contrôles sont conformes (taille, aucune signature, aucune partition, non monté, différent du disque système). Système de fichiers ext4 monté sur `/data` par UUID avec `nofail`.

— **Mise à jour contrôlée** : snapshot, pré-contrôles, simulation `apt-get -s upgrade`, mise à jour sans suppression de paquet, contrôle de l'historique apt et du besoin de redémarrage.

4.3.2 Résultats

Table 4.7 : Phase C — vérifications et résultats

Vérification	Résultat observé	Preuve
Permissions	alice écrit dans le site, chloe est refusée ; chloe lit le journal et `/data/rapports`, alice est refusée	P-20, P-22
Service web	Actif et activé, port 80 en écoute, HTTP 200 depuis OPS01 ; arrêt → HTTP 000, redémarrage → nouveau processus maître	P-21
Stockage	Contrôles conformes, `/dev/sdb` ext4 monté sur `/data` par UUID, remontage par fstab réussi	P-22
Mises à jour	20 paquets mis à jour, code retour 0, pas de redémarrage requis	P-23
Journaux	Tentative SSH « pirate » refusée, requête 404 tracée, ajout à chaud du disque tracé, aucune erreur	P-24

Difficulté : chemin de disque deviné

Les premiers contrôles avant formatage visaient un chemin `/dev/disk/by-id/...` supposé, qui n'existait pas. Aucun formatage n'avait lieu dans ce premier script. Correction : lecture du nom réel et arrêt automatique si un contrôle n'est pas conforme. Le formatage a ainsi visé uniquement le bon disque.

Retour arrière : snapshot `s02_avant_maj_c` (disque de test compris), sauvegarde de la configuration réseau dans `/root/`.

Chapitre 5

Réalisation — phases D à H

5.1 Phase D — Automatisation avec Ansible

Objectif

Fournir des playbooks Linux et Windows reproductibles et idempotents.

Responsables : Kenzi Bendjelloul · **Dates :** 19/10/2026 · **Statut :** Fait

5.1.1 Réalisation

Ansible est installé sur OPS01 avec les collections du dépôt (`ansible/requirements.yml`). Un compte de service `ansible` dédié (clé ed25519, sudo sans mot de passe validé par `visudo -c`) administre LINUX01 par SSH ; WIN-DC01 est administré par WinRM (HTTP 5985, NTLM) avec le mot de passe Administrateur lu dans un **vault chiffré AES256**.

TABLE 5.1 : Playbooks du dépôt

Playbook	Cible	Actions	Vérifications intégrées
`windows_adds.yml`	WIN-DC01	Rôle AD DS, forêt, NTP, redirecteurs, client DNS	Services AD, état du domaine
`windows_base.yml`	WIN-DC01	OU, groupes, comptes, partage SMB et ACL NTFS, zone inverse, enregistrements A/PTR, windows_exporter 0.31.8, pare-feu 9182 limité à OPS01	Services critiques démarrés, `/metrics` joignable
`linux_base.yml`	LINUX01	Paquets, comptes, droits, site nginx (modèle + rechargement), `/data` par UUID, pare-feu UFW, node_exporter, contrôle planifié (séance 6)	HTTP 200, contenu publié, `/metrics`
`ops01_monitoring.yml`	OPS01	Prometheus, règles, Grafana provisionné	Toutes les cibles `up`, compte admin Grafana
`linux_maintenance.yml`	Linux	Maintenance (phase E)	Pré- et post-contrôles
`windows_maintenance.yml`	WIN-DC01	Windows Update (phase E)	Pré- et post-contrôles
`controle_sante.yml`	3 serveurs	Aucune : contrôle de santé en lecture seule (séance 6)	Tableau OK / ATTENTION / ECHEC
`linux_quotas.yml`	LINUX01	Quotas sur le disque de test (séance 6)	Garde-fou sur le numéro de série

Deux contrôles ont été ajoutés le 20/10 par le chef de projet : chaque playbook Windows et la supervision commencent par vérifier que les variables du vault sont disponibles, *avant toute modification* ; le script `scripts/verif-idempotence.sh` enchaîne deux exécutions et renvoie 0 si le second passage est à `changed=0`, 1 s'il reste des changements, 2 en cas d'échec.

5.1.2 Résultats

TABLE 5.2 : Phase D — vérifications et résultats

Vérification	Résultat observé	Preuve
Inventaire et connexions	3 hôtes, 3 groupes ; `ping: pong` et `win_ping: pong`	**P-30**
1er passage	Linux : 22 tâches OK dont 10 changements ; Windows : 21 tâches OK dont 12 changements ; aucun échec	**P-31**, **P-33**
2e passage (idempotence)	Linux `changed=0`, Windows `changed=0`	**P-32**, **P-34**
Script d'idempotence	Code 0 sur les playbooks réels, 1 sur un playbook non idempotent, 2 sur un playbook en échec	**P-61**, **P-62**
Contrôle du vault	Arrêt avant toute tâche si le vault est absent ou invalide	**P-62**

5.1.3 Difficultés et décisions

TABLE 5.3 : Phase D — difficultés rencontrées

Problème	Cause	Correction et résultat
L'option MSI de windows_exporter ouvre le port 9182 à tous	Comportement de l'installeur	Option retirée, règle dédiée limitée à OPS01
Le PTR du DC repasse en `changed`	Le DC réenregistre lui-même son PTR (dynamique, TTL 20 min) ; le playbook le réécrivait en statique : chacun écrasait l'autre	Le playbook déclenche seulement l'enregistrement du DC s'il manque : `changed=0` même après réenregistrement forcé (**P-36b**, **P-37**)
Test négatif du vault exécuté réellement	`--vault-password-file` s'ajoute au fichier configuré au lieu de le remplacer	Impact vérifié (valeur inchangée) ; test refait avec `ANSIBLE_VAULT_PASSWORD_FILE` ; défaut d'idempotence révélé puis corrigé (**P-62**, **P-15**)
1er pipeline CI en échec	Dashboard absent, ligne trop longue, 7 remarques ansible-lint	Corrigé ; mêmes outils installés en local pour valider avant de pousser

5.2 Phase E — Maintenance et mises à jour

Objectif

Formaliser et tester une maintenance sûre et traçable sur les deux systèmes.

Responsables : MIR Mahan, Jolan Allen · **Dates :** 19/10/2026 · **Statut :** Fait

La procédure (`docs/procedure-maintenance.md`) fixe la fenêtre, l'ordre (LINUX01, puis OPS01, puis WIN-DC01 en dernier car tout le laboratoire en dépend), les précautions et le retour arrière. Elle est outillée par deux playbooks qui **refusent de continuer** si un pré-contrôle échoue.

Table 5.4 : Étapes de la maintenance outillée

Étape	Linux (`linux_maintenance.yml`)	Windows (`windows_maintenance.yml`)
Retour arrière	Snapshot `s04_avant_maintenance_e`	Snapshot `s04_avant_maj_windows`
Pré-contrôles	Systèmes de fichiers < 85 % ; services attendus actifs	Volumes fixes < 85 % ; services AD démarrés ; aucun redémarrage en attente
Mises à jour	`unattended-upgrades` (sécurité quotidienne) ; `apt upgrade` sans suppression	Recherche, puis installation seulement avec `-e maintenance_apply=true`
Redémarrage	Seulement s'il est requis **et** autorisé (`-e allow_reboot=true`)	Seulement avec `-e allow_reboot=true`
Post-contrôles	Services actifs ; site HTTP 200	Services critiques ; domaine et DNS ; historique Windows Update
Trace	Rapport par serveur dans `~/infraready-logs/`	Idem + journal Windows Update

Table 5.5 : Phase E — vérifications et résultats

Vérification	Résultat observé	Preuve
Mise à jour Linux	OPS01 mis à jour ; aucun redémarrage requis ; mises à jour de sécurité automatiques actives	**P-40**, **P-43**
Mise à jour Windows	5 mises à jour installées en 1 h 13 avec redémarrage du DC (cumulative KB5126646, .NET, MSRT, Defender) ; passage complet après exclusion de KB5007651	**P-41**, **P-42**, **P-42c**
Capacité disque	Tous les volumes < 85 % ; `C:` de 24,2 % à 37,2 % après la cumulative	**P-40b**, **P-42c**
Services	Tous actifs avant et après ; site HTTP 200	**P-40**, **P-40b**

Difficulté : boucle Windows Update sur KB5007651

L'installation se termine en échec après 1 h 13 alors que cinq mises à jour sont installées : la plateforme Windows Security KB5007651 est marquée « installée » trois fois de suite mais toujours reproposée, et le module `win_updates` arrête la tâche. Diagnostic par l'historique Windows Update, le journal Système et `Get-MpComputerStatus` (plateforme bien à jour). Décision : exclusion de KB5007651 (`reject_list`), à suivre manuellement ; passage suivant complet, `failed=0` (**P-42** → **P-42b** → **P-42c**). Autre difficulté corrigée : un lecteur CD vide était vu comme un volume plein à 100 % (**P-41a**).

5.3 Phase F — Supervision avec Prometheus et Grafana

Objectif

Collecter et visualiser les métriques réelles des serveurs et démontrer une alerte.

Responsables : MIR Mahan, Jolan Allen · **Dates :** 19/10/2026 · **Statut :** Fait

5.3.1 Réalisation

Tout est déployé par `ops01_monitoring.yml` à partir des fichiers versionnés dans `monitoring/` : configuration et règles de Prometheus (validées par `promtool` avant tout redémarrage), source de données et tableau de bord Grafana provisionnés, mot de passe administrateur Grafana lu dans le vault, accès anonyme en lecture seule. Les exporters sont installés par `linux_base.yml` et `windows_base.yml` ; leurs ports ne sont ouverts qu'à OPS01 (figure 5.1).

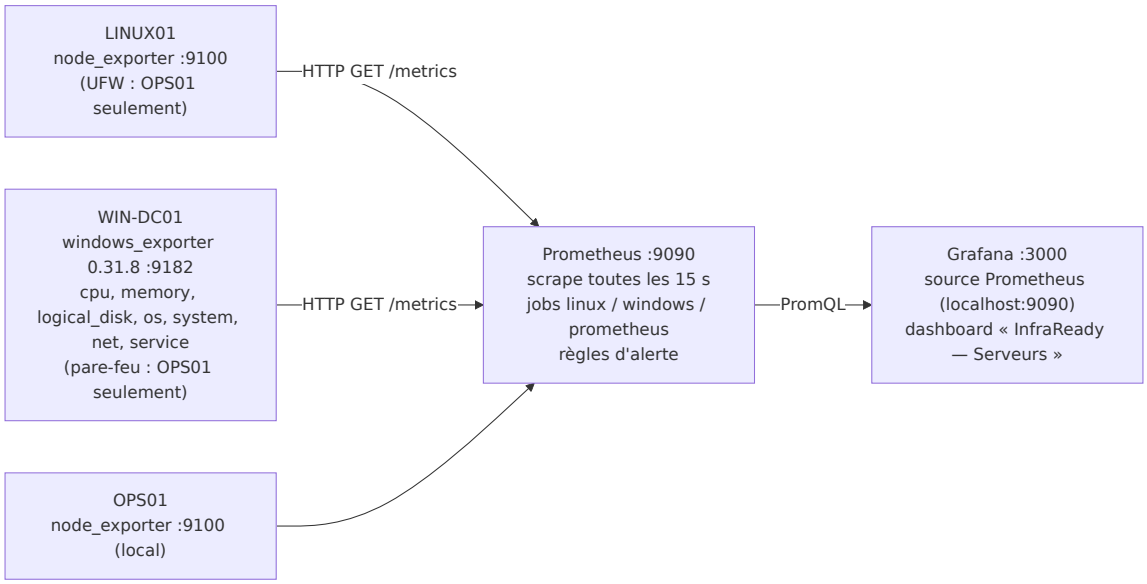


Figure 5.1 : Origine, ports et cheminement des métriques (phase F)

TABLE 5.6 : Règles d'alerte (5 en phase F, 7 après la séance 6)

Alerte	Condition	Durée	Sévérité
`InstanceDown`	Cible Linux ou Windows injoignable (`up == 0`)	1 min	critical
`ServiceWindowsCritiqueArrete`	Service DNS, NTDS, Netlogon ou Kdc arrêté	1 min	critical
`CpuEleve`	CPU > 80 %	2 min	warning
`MemoireElevee`	Mémoire > 90 %	5 min	warning
`DisquePresquePlein`	Volume > 85 %	5 min	warning
`SiteWebIndisponible` (séance 6)	Contrôle planifié du site en échec	2 min	critical
`ControlePlanifieEnRetard` (séance 6)	Contrôle planifié non exécuté depuis 5 min	5 min	warning

5.3.2 Résultats

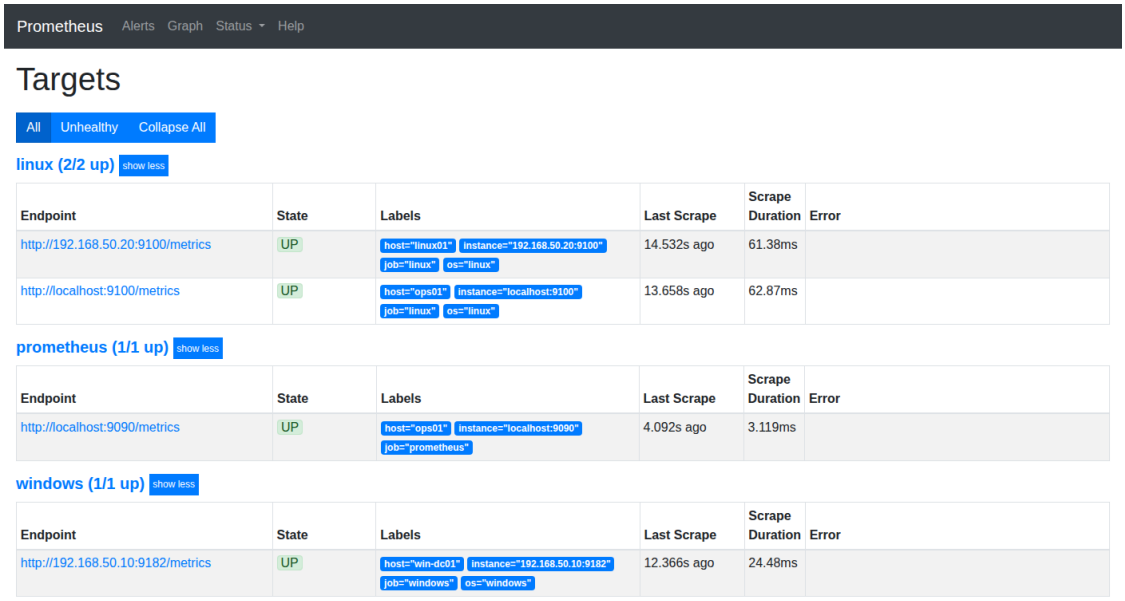


FIGURE 5.2 : Cibles Prometheus : 4/4 `up` (**P-51**)

TABLE 5.7 : Phase F — vérifications et résultats

Vérification	Résultat observé	Preuve
Collecte Linux	LINUX01 `up` (1 709 métriques) ; CPU 0,2 %, mémoire 11,3 %, `/` 29,1 %	**P-51**, **P-52**
Collecte Windows	WIN-DC01 `up` (3 263 métriques) ; CPU 1,0 %, mémoire 26,7 %, `C:` 24,2 % ; services AD à 1	**P-51**, **P-52**
Tableau de bord	3 serveurs « EN LIGNE », CPU, mémoire, stockage, alertes ; version 2 avec filtre d'hôte (PR #3)	**P-53**, **P-56**
Alerte démontrée	`InstanceDown` pending à +10 s, firing à +60 s, résolue 16 s après le retour du service	**P-54**, **P-55**
Idempotence	`ops01_monitoring.yml` : `changed=0` au 2e passage	**P-50b**

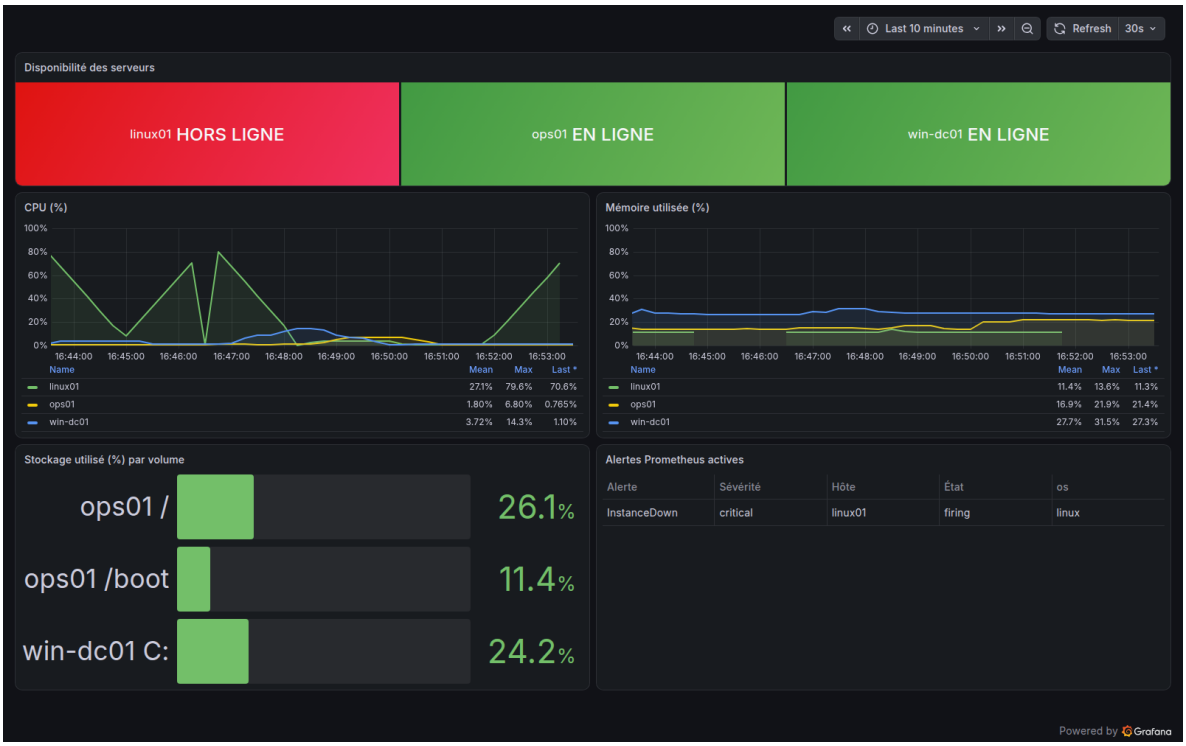


FIGURE 5.3 : Dashboard Grafana (version 1) pendant la démonstration : LINUX01 hors ligne et alerte active (**P-55**)

Difficulté : mot de passe Grafana tronqué

Après le premier passage, le compte administrateur Grafana était refusé : le mot de passe contenait « # », lu comme un début de commentaire dans `grafana.ini`. Correction : mot de passe sans caractère ambigu, contrôle `assert` dans le playbook et vérification sur l'API d'administration (**P-50a** → **P-50**).

5.4 Phase G — GitHub et intégration continue

Objectif
Versionner les livrables et valider le dépôt automatiquement.
Responsables : MIR Mahan, Jolan Allen · **Dates :** 19–23/10/2026 · **Statut :** Fait

Le dépôt `jolanallen/InfraReady` regroupe playbooks, scripts, supervision, modèles d'installation, documentation et preuves (annexe B). Le workflow `.github/workflows/validation.yml` s'exécute à chaque push et à chaque demande de fusion.

TABLE 5.8 : Jobs du pipeline GitHub Actions

Job	Contrôles
Arborescence attendue	Présence des livrables (README, inventaire, playbooks, supervision)
YAML + Ansible	`yamllint --strict`, `ansible-playbook --syntax-check` de chaque playbook, `ansible-lint`
Prometheus + Grafana	`promtool check config` (configuration et règles), JSON du dashboard valide
Aucun secret en clair	gitleaks sur tout l'historique ; fichiers vault obligatoirement chiffrés

— Premier run en échec (dashboard absent, lint) puis vert sur le dépôt de travail (**P-60a**) ; historique migré le 20/10 vers le dépôt de l'équipe, run vert (**P-60**).

— Depuis le 20/10, toute modification passe par une branche et une demande de fusion (16 au 23/10), relue et fusionnée par le chef de projet ; la CI est verte sur chacune (**P-61** à **P-64**).

— **Limite assumée** : la CI valide les fichiers mais ne déploie rien ; un runner hébergé par GitHub ne peut pas joindre le réseau privé du laboratoire. Le déploiement se fait depuis OPS01.

5.5 Phase H — Diagnostic d'incident

Objectif
Reproduire un incident contrôlé, en trouver la cause et prouver le retour à la normale.
Responsables : MIR Mahan, Jolan Allen · **Dates :** 19/10/2026 · **Statut :** Fait

Incident INC-2026-10-19-01. À 19:27, la directive `root` du site nginx de LINUX01 est modifiée vers un répertoire inexistant (« mise en production » simulée, sauvegarde préalable). `nginx -t` ne vérifie pas l'existence du répertoire : le rechargement réussit et le site répond 404 pour toutes les pages.

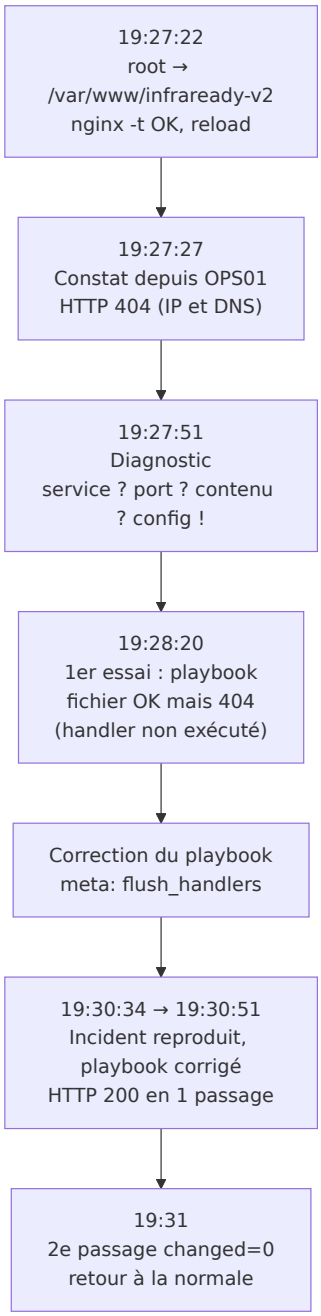


FIGURE 5.4 : Chronologie de l'incident de la phase H

TABLE 5.9 : Phase H — démarche et résultats

Étape	Constat	Preuve
Symptôme	HTTP 404 depuis OPS01 par adresse et par nom ; VM joignable, port 80 ouvert	**P-70**
Diagnostic	Trois hypothèses écartées (service, pare-feu, contenu) ; `nginx -T` révèle le mauvais `root` ; `stat`, `diff` et le journal sudo datent et attribuent le changement	**P-71**
Correction	Rejeu de la configuration de référence par `linux_base.yml` plutôt qu'une correction manuelle	**P-72a**, **P-72**
Retour à la normale	HTTP 200 par adresse et par nom ; 2e passage `changed=0`	**P-73**

Défaut révélé dans le playbook

La première correction restaurait bien le fichier mais le site restait en 404 : la vérification HTTP échouait avant la fin du play, et Ansible n'exécute les gestionnaires (ici, le rechargement de nginx) qu'en fin de play. Correction : `meta: flush_handlers` avant les vérifications. L'incident a été reproduit une seconde fois pour prouver que le playbook corrigé le résout en un seul passage. Rapport complet : `docs/rapport-incident.md`.

Chapitre 6

Séance 4 — Stockage, processus et scénarios de panne

Objectif

Appliquer au laboratoire le TP de la séance 4 : disque de test identifié et monté de façon persistante, processus et services observés sans perturbation, scripts de vérification à codes de sortie, playbook commenté rejoué deux fois, six scénarios de panne contrôlée avec une procédure commune de diagnostic.

Responsables : MIR Mahan, Jolan Allen, Kenzi Bendjelloul · **Dates :** 19–20/10/2026 · **Statut :** **Fait** (bilans individuels à rédiger)

6.1 Correspondance avec le support

TABLE 6.1 : Étapes du support et réalisation

Étape du support	Réalisation dans InfraReady	Preuves
Disque identifié, formaté, monté, droits	Disque de 10 Go identifié par son numéro de série `INFRA-DATA01`, ext4 monté sur `/data` par UUID, droits par groupe	**P-22**
État initial Linux et Windows	Processus, mémoire, disques, services et journaux relevés sans arrêter de service critique	**P-14**, **P-21**, **P-26**
Scripts de vérification	`check-nginx.sh` et `Check-DnsClient.ps1`, testés dans trois situations	**P-25**, **P-14**
Playbook commenté	`linux_base.yml` complété (message publié et contrôle du contenu) : `changed=1` puis `changed=0` ; simulation `--check` corrigée	**P-38**, **P-38b**, **P-39**
Six scénarios de panne	Un seul défaut à la fois, retour à l'état initial vérifié	**P-74** à **P-79**

6.2 Scripts de vérification

Les deux scripts sont en lecture seule : ils diagnostiquent sans rien corriger, et leurs codes de sortie sont exploitables par un autre script.

TABLE 6.2 : Codes de sortie des scripts de vérification

Script	Code 0	Code 1	Code 2
`check-nginx.sh`	Service actif, HTTP 200, texte attendu présent	Service inactif, site injoignable, HTTP différent de 200 ou texte absent	Commande absente, service inconnu
`Check-DnsClient.ps1`	Service démarré	Service présent mais arrêté	Service introuvable

6.3 Scénarios de panne contrôlée

TABLE 6.3 : Six scénarios de panne contrôlée

#	Défaut introduit	Symptôme, cause et correction	Preuve
1	Variable `site_message` incorrecte	HTTP 200 et playbook sans erreur, mais message incorrect publié ; valeur restaurée, contrôles `--syntax-check`, `--list-hosts`, `--check --diff`	P-74
2	Variable Jinja2 mal orthographiée	Tâche de modèle en échec (« undefined ») ; l'ancienne page reste servie	P-75
3	Mauvais chemin de destination	Fichier créé dans `/tmp`, page servie inchangée ; `nginx -T` confirme la racine réelle	P-76
4	Notification du gestionnaire retirée	Fichier modifié sans rechargement de nginx : l'en-tête ajouté n'est pas servi ; rétablir la notification ne suffit pas sans nouvelle modification	P-77
5	nginx arrêté	`check-nginx.sh` renvoie 1, curl en échec ; redémarrage puis script à 0 et playbook `changed=0`	P-78
6	Contenu publié incomplet	HTTP 200 mais l'assertion « message attendu publié » échoue ; modèle restauré	P-79

6.4 Enseignements

— **Un code HTTP 200 ne prouve pas le bon contenu** : d'où le contrôle du contenu ajouté au playbook (scénarios 1 et 6).

— **Le mode `--check` a des limites** : les tâches de lecture (`blkid`, `uri`) n'y sont pas exécutées, ce qui produisait un faux changement sur `/data` ; elles sont désormais marquées `check_mode: false` (P-39a → P-39).

— **Un pic de CPU ne prouve pas un défaut** : le processus en tête de liste était lancé par notre propre commande de relevé (P-26).

— **Le module `script` d'Ansible ramène tout code non nul à 1** : le code 2 du script PowerShell n'est observable qu'en exécution directe (P-14).

Retour arrière : snapshot `s07_avant_seance4` des trois VM et sauvegardes `.etat-initial` des

fichiers du projet pendant les scénarios.

Chapitre 7

Séance 6 — Maintenance opérationnelle et supervision

Objectif

En tant que techniciens d'exploitation, contrôler l'état de la plateforme, fiabiliser la maintenance, rendre les anomalies observables et conserver des preuves vérifiables : socle obligatoire UC-01 à UC-06, compléments UC-C1 à UC-C4 et un défi avancé (UC-A3).

Responsables : MIR Mahan, Jolan Allen, Kenzi Bendjelloul · **Dates :** 23/10/2026 · **Statut :** **Fait** — UC-05 **Partiel**

7.1 Déroulé et bilan

La séance a commencé par un blocage : l'hôte Proxmox avait été redémarré et les VM, sans démarrage automatique, étaient éteintes. Une fois les VM démarrées, un snapshot `s09_avant_seance6` a été pris avant toute modification (figure 7.1).

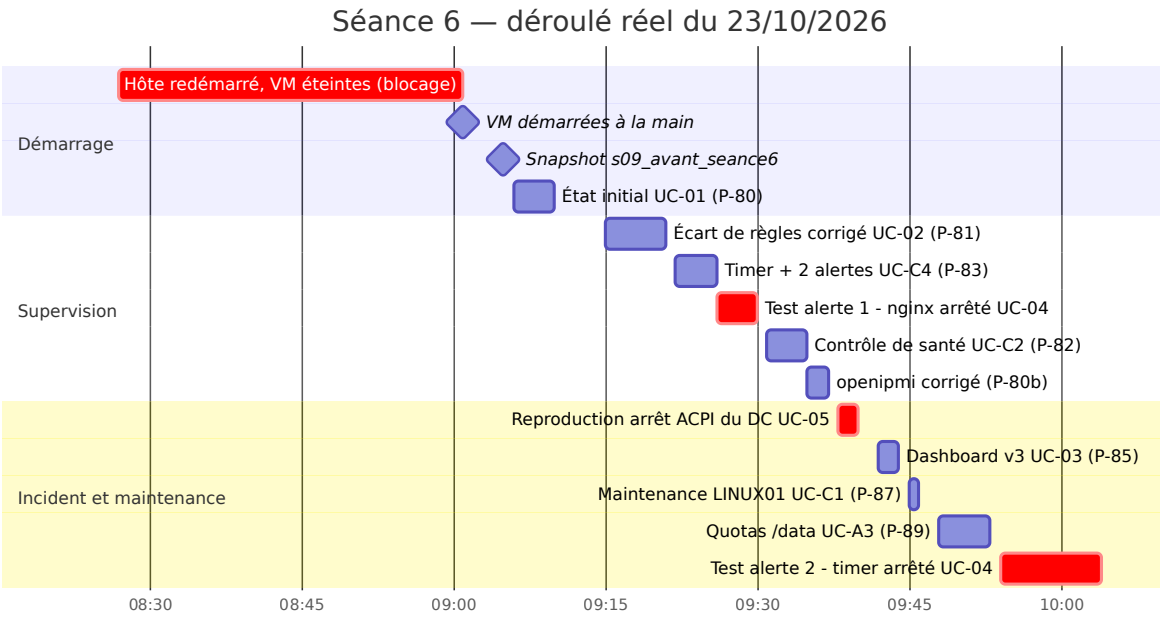


FIGURE 7.1 : Déroulé réel de la séance 6 (en rouge : blocage et tests de panne contrôlés)

TABLE 7.1 : Bilan de la séance 6 par use case

Use case	Réalisation	Statut	Preuves
UC-01 État de santé	Fiche d'état initial datée avant toute modification ; 5 écarts relevés	**Fait**	**P-80**, **P-80c**
UC-02 Métriques	4/4 cibles collectées ; écart de règles et vérification du playbook corrigés	**Fait**	**P-81**
UC-03 Tableau de bord	Dashboard v3 : unités, descriptions, données absentes distinguées de zéro	**Fait**	**P-85**
UC-04 Alerte	2 alertes déclenchées puis résolues par des pannes contrôlées	**Fait**	**P-84**
UC-05 Incident	Arrêt brutal du DC : cause confirmée, correction préparée	**Partiel**	**P-86**
UC-06 Livrables	Code, CI, README, dossier ; validé / partiel / bloqué listé	**Fait**	**P-64**
UC-C1 Maintenance	Maintenance de LINUX01 entre deux contrôles de santé	**Fait**	**P-87**
UC-C2 Contrôle Ansible	`controle_sante.yml` en lecture seule	**Fait**	**P-82**
UC-C3 Tendance	Mesure ponctuelle comparée à 4 jours d'historique	**Fait**	**P-88**
UC-C4 Tâche planifiée	Timer systemd tracé dans le journal et dans Prometheus	**Fait**	**P-83**
UC-A3 Quotas (défi)	Quotas sur le disque de test `/data`	**Fait**	**P-89**

7.2 UC-01 — État de santé vérifiable

Le relevé initial (09:06) couvre l'identité, l'heure, les disques, les services, les unités en échec, les erreurs récentes et un test fonctionnel par service (résolution DNS, page HTTP avec contrôle du contenu, API Prometheus et Grafana, requête Active Directory). Sous Linux, il utilise `hostnamectl`, `timedatectl`, `df`, `systemctl`, `journalctl`, `getent` et `curl` ; sous Windows, un script PowerShell en lecture seule exécuté par WinRM.

TABLE 7.2 : Écarts relevés par l'état initial et suite donnée

#	Écart	Cause et suite	Statut
1	VM éteintes après le redémarrage de l'hôte	Pas de démarrage automatique ; VM démarrées à la main	Levé
2	Jeton API Proxmox sans droit	Séparation des privilèges ; retirée par l'étudiant	Levé
3	4 règles d'alerte chargées au lieu de 5	Copie du projet sur OPS01 non resynchronisée ; supervision redéployée	Corrigé
4	`openipmi.service` en échec sur les VM Linux	Paquet recommandé installé par nos playbooks, sans matériel IPMI ; service désactivé par Ansible	Corrigé
5	WIN-DC01 : événements 41 et 6008	Arrêt non propre à chaque arrêt de l'hôte : incident UC-05	Diagnostiqué

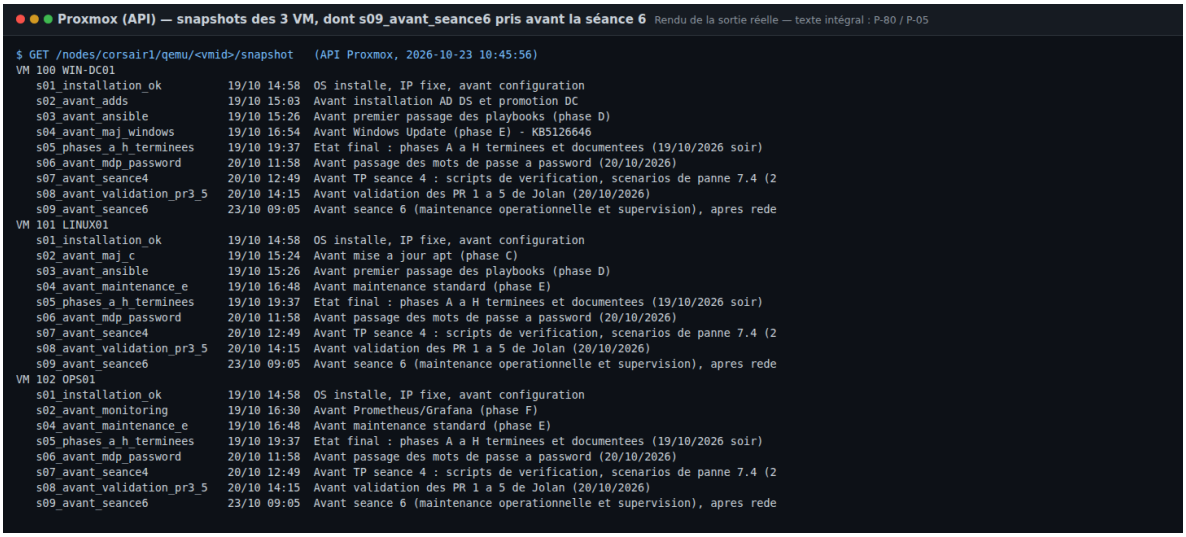


FIGURE 7.2 : Snapshots des trois VM, dont `s09_avant_seance6` (rendu de la sortie de l'API, **P-80c**)

7.3 UC-02 — Métriques Linux et Windows collectables

La collecte a été vérifiée de bout en bout : configuration chargée, connexions TCP établies vers les exporters, endpoints `/metrics` en HTTP 200, puis valeurs lues dans la base de Prometheus (figure 7.3).

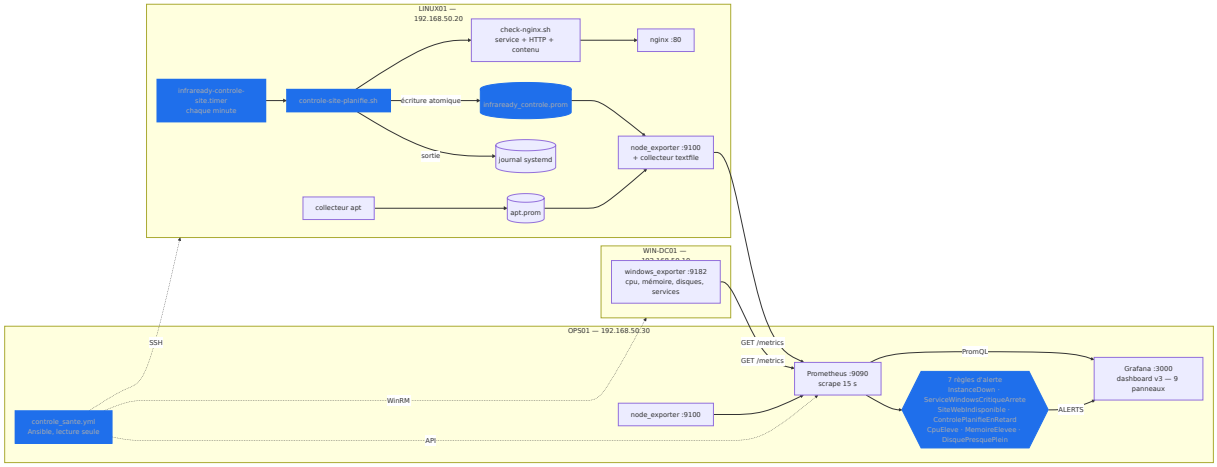


FIGURE 7.3 : Architecture de supervision après la séance 6 (en bleu : éléments ajoutés)

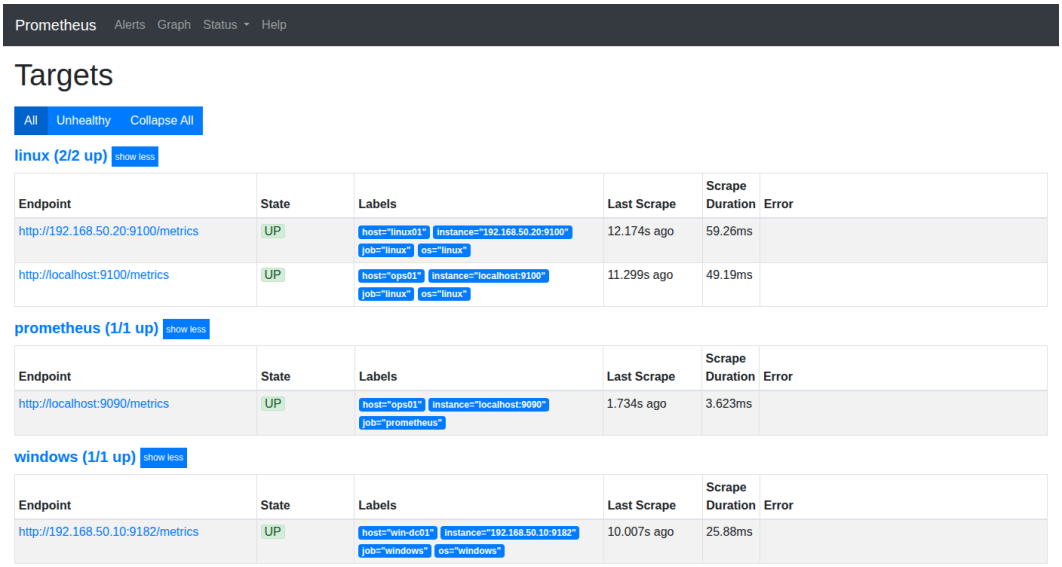


FIGURE 7.4 : Prometheus : 4/4 cibles up après correction (**P-81**)

Défaut révélé : vérification des cibles qui réussissait sans cible

Juste après un redémarrage de Prometheus, la liste des cibles est vide : l'assertion du playbook réussissait alors sans rien vérifier (« 0 cibles up »). Au passage suivant, une cible encore « unknown » la faisait échouer. Correction : attendre le premier scrape (8 tentatives de 5 s) et exiger une liste non vide ; le cas défaillant a été rejoué avec succès (**P-81a** → **P-81b**).

7.4 UC-03 — Tableau de bord d'exploitation

Le dashboard « InfraReady — Serveurs » passe en version 3 : description d'usage, période de 6 heures, une description « ce que le panneau permet de décider » sur chacun des 9 panneaux, des unités explicites, et surtout la distinction entre une **donnée absente** (« NON COLLECTÉ », courbes interrompues) et une **valeur réellement nulle**. Trois panneaux sont ajoutés : contrôle planifié du site, maintenance Linux et évolution du stockage.



FIGURE 7.5 : Dashboard Grafana v3 : le laboratoire était éteint avant 9 h, d'où les zones vides plutôt que des zéros (**P-85**)

7.5 UC-04 et UC-C4 — Tâche planifiée et alertes testées

L'incident de la phase H avait montré qu'un site indisponible ne déclenchait aucune alerte, puisque seul l'exporter était surveillé. Un **timer systemd** lance désormais `check-nginx.sh` chaque minute sur LINUX01 et publie le résultat pour Prometheus via le collecteur « textfile » de node_exporter.

Le service tourne en root **sans aucune capacité**, avec un système de fichiers en lecture seule sauf le dossier des métriques.

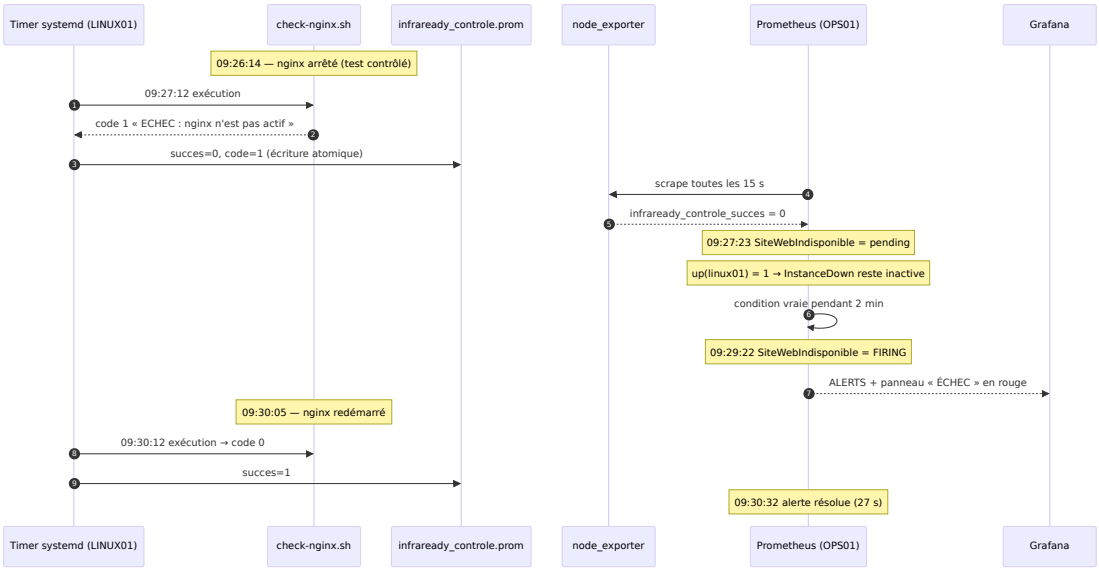


FIGURE 7.6 : Chaîne de détection pendant le test 1 (nginx arrêté)

TABLE 7.3 : Les deux alertes ajoutées

	`SiteWebIndisponible`	`ControlePlanifieEnRetard`
Condition	Contrôle du site en échec (`infraready_controle_succes == 0`)	Dernier contrôle de plus de 300 s, ou métrique absente
Durée, gravité	2 min (deux contrôles consécutifs), critical	5 min, warning
Première action	Journal du contrôle, puis état de nginx	État du timer, puis journal du contrôle
Test contrôlé	nginx arrêté de 09:26 à 09:30	Timer arrêté de 09:54 à 10:04
Résultat	Firing à +3 min 08 ; `InstanceDown` reste inactive ; résolue 27 s après	Firing à 10:04:01 ; résolue à 10:04:17

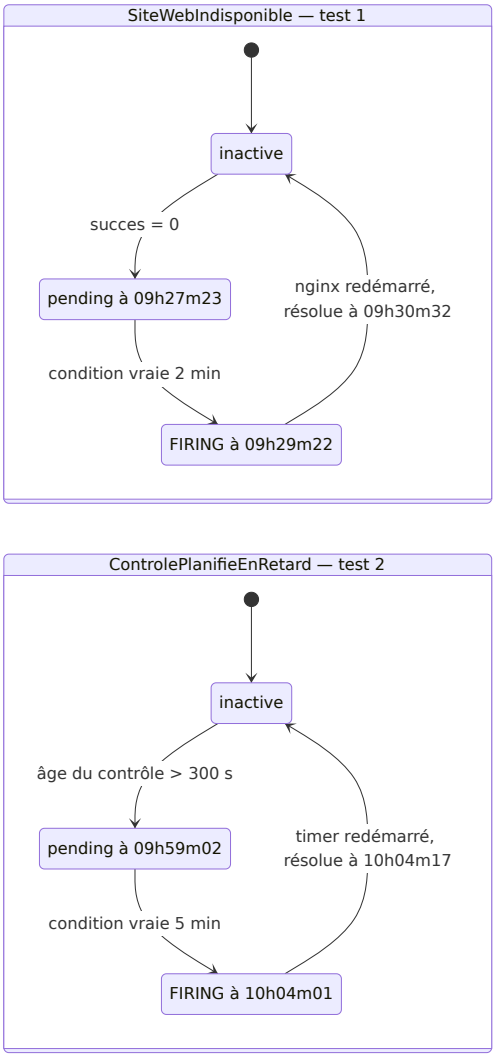


FIGURE 7.7 : Cycle de vie des deux alertes pendant les tests



FIGURE 7.8 : Dashboard figé à 09:29:40 : contrôle du site en ÉCHEC et alerte `SiteWebIndisponible` active, alors que LINUX01 reste EN LIGNE (**P-84**)

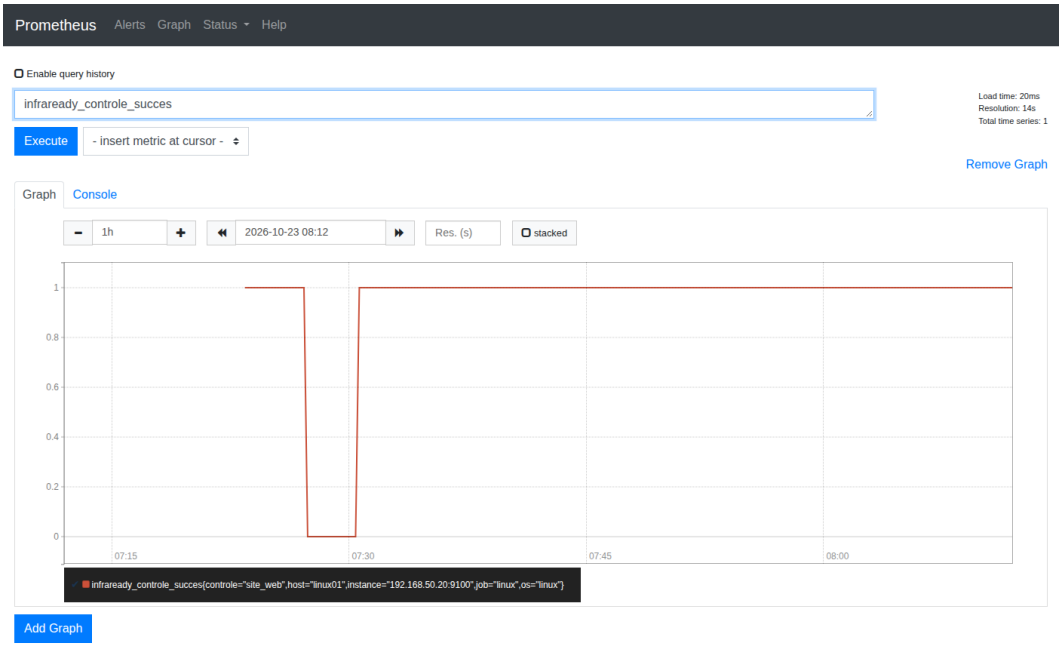


FIGURE 7.9 : Prometheus : `infraready_controle_succes` tombe à 0 pendant l'arrêt de nginx (axe horaire en UTC, soit l'heure de Paris moins 2 h)



FIGURE 7.10 : Timer, journal du contrôle et métriques publiées sur LINUX01 (rendu de la sortie, **P-83**)

7.6 UC-05 — Incident : arrêt brutal du contrôleur de domaine

Symptôme. Au démarrage du 23/10, WIN-DC01 journalise les événements 41 (Kernel-Power) et 6008 (« l'arrêt précédent du système était inattendu »). Le même couple d'événements existe après l'arrêt du 19/10 : l'incident est systématique. **Impact supposé :** risque d'incohérence de la base Active Directory et des zones DNS à chaque arrêt de l'hôte.

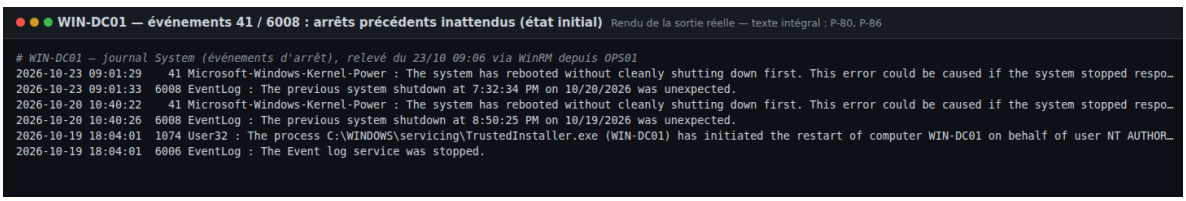


FIGURE 7.11 : Événements 41 et 6008 de WIN-DC01 (rendu de la sortie, **P-80**)

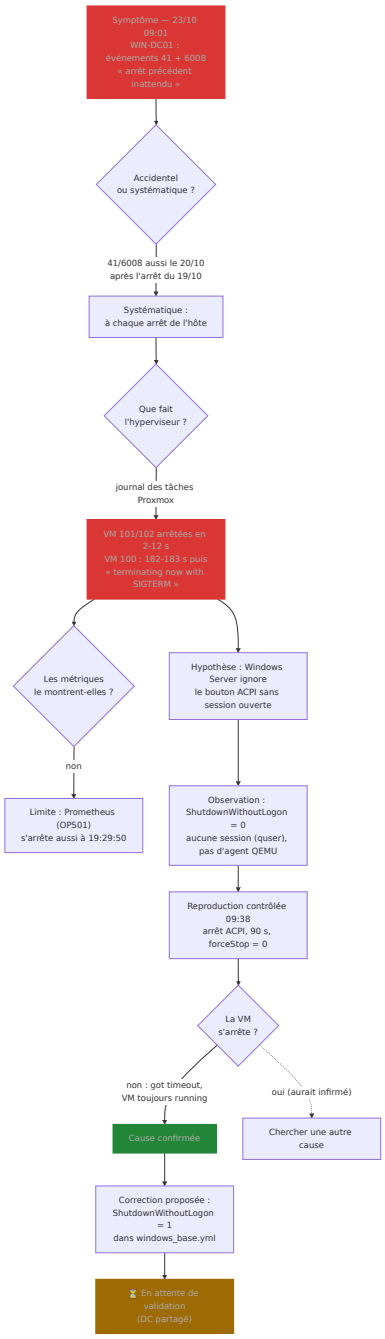


FIGURE 7.12 : Démarche de diagnostic de l'incident INC-2026-10-23-01

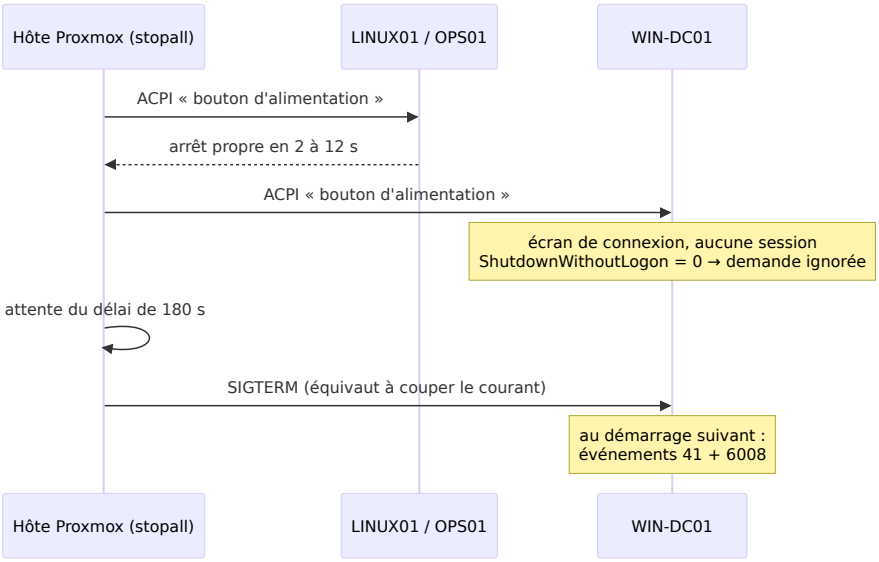


FIGURE 7.13 : Ce qui se passe à l'arrêt de l'hôte



FIGURE 7.14 : Journal des tâches Proxmox : la VM 100 tuée après 182–183 s, puis la reproduction contrôlée qui échoue sur délai (rendu de la sortie, **P-86**)

— **Cause confirmée :** Windows Server ignore la demande d'arrêt ACPI à l'écran d'ouverture de session lorsque la stratégie `ShutdownWithoutLogon` vaut 0, ce qui est sa valeur par défaut ; Proxmox attend 180 s puis tue la VM.

— **Limite mise en évidence :** Prometheus tourne sur OPS01, arrêtée en même temps : la supervision ne peut pas observer cet incident.

— **Correction proposée :** passer `ShutdownWithoutLogon` à 1 par `windows_base.yml` (retour arrière : valeur 0), puis valider par un arrêt ACPI réel et l'absence d'événements 41/6008.

Correction non appliquée

La correction modifie le contrôleur de domaine partagé : elle attend une validation explicite de l'équipe. Rapport complet : `docs/rapport-incident.md` (INC-2026-10-23-01).

7.7 UC-06 — Intégration aux livrables

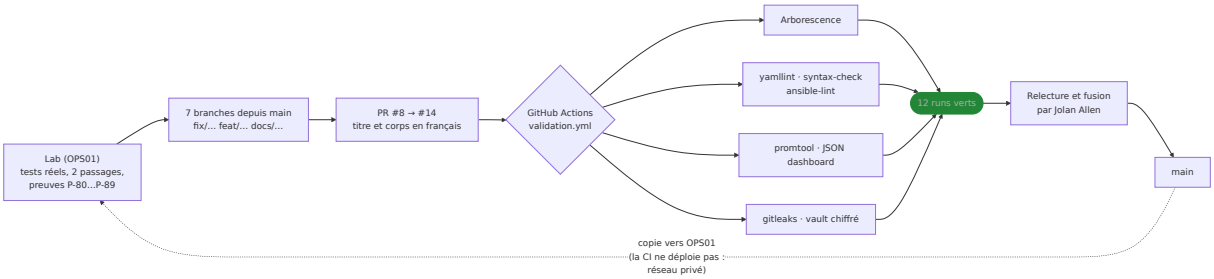


FIGURE 7.15 : Du laboratoire au dépôt : branches, demandes de fusion, CI et fusion

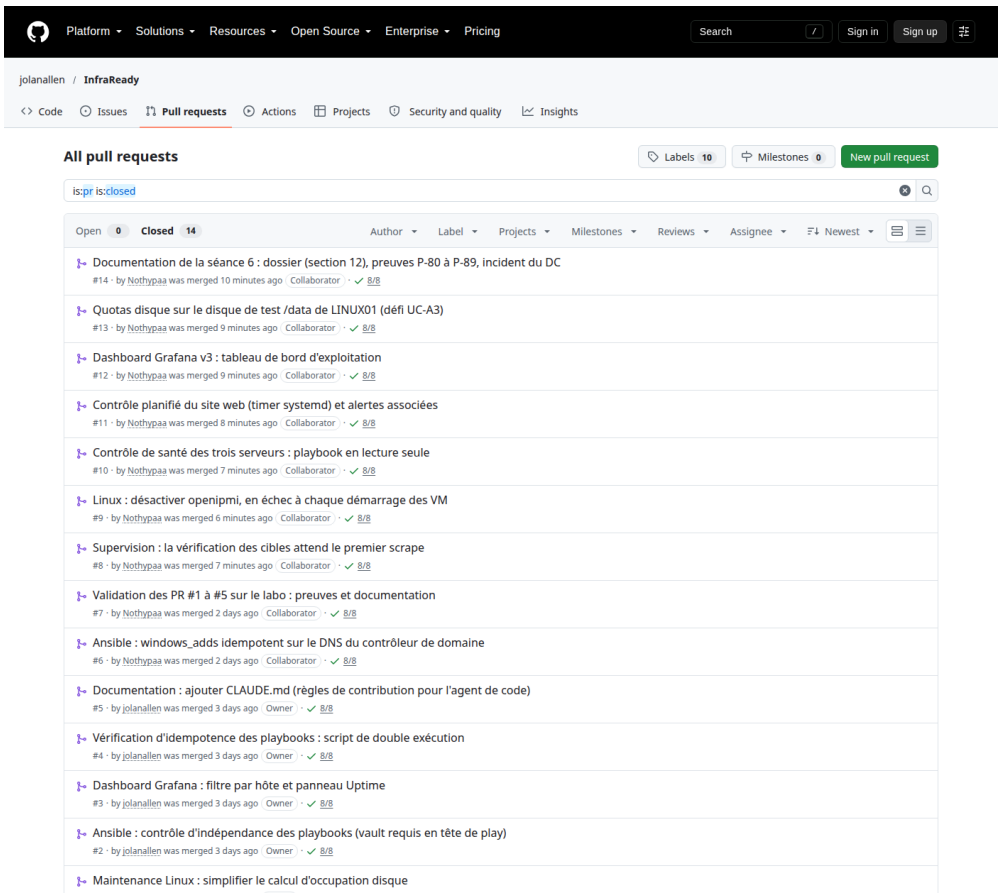


FIGURE 7.16 : Demandes de fusion de la séance 6, fusionnées avec des contrôles CI 8/8 (**P-64**)

7.8 UC-C1 et UC-C2 — Maintenance encadrée par un contrôle de santé

Le playbook `controle_sante.yml` contrôle les trois serveurs en lecture seule (`changed=0`, y compris en `--check`) et produit un tableau OK / ATTENTION / ECHEC / NON APPLICABLE / INACCESSIBLE ; il échoue (code 2) s'il reste un échec, ce qui permet de l'utiliser comme critère d'arrêt d'une maintenance.

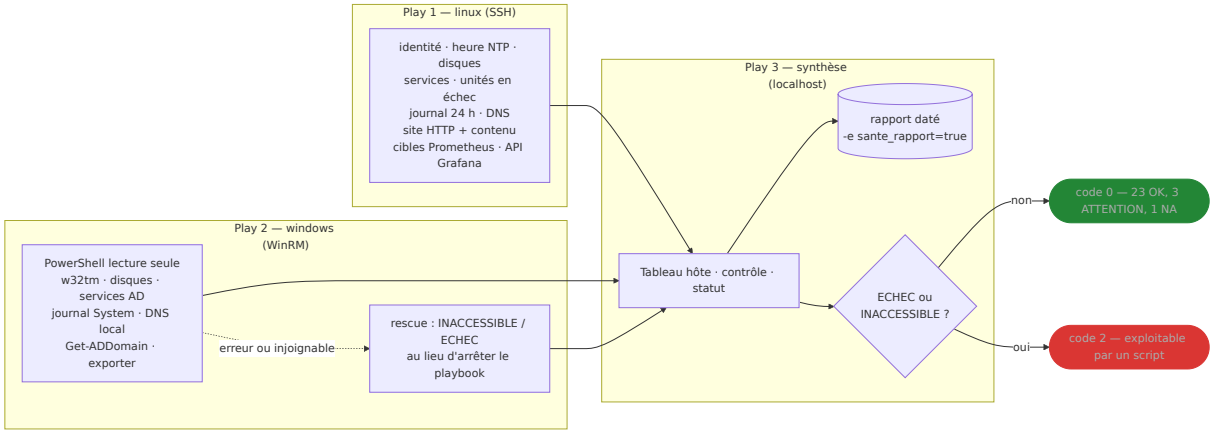
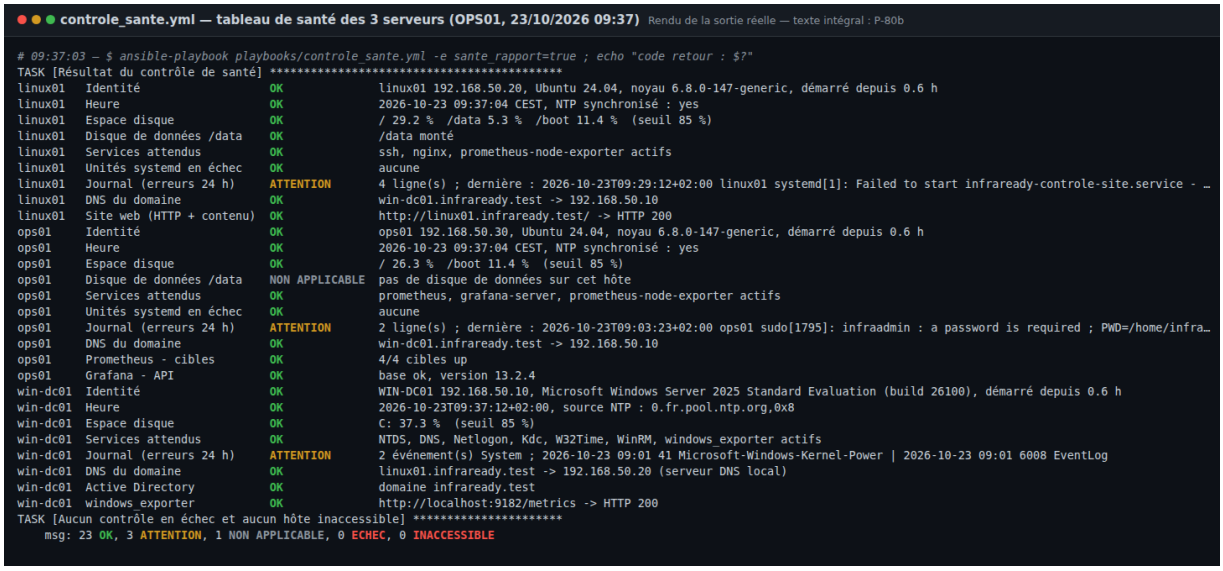


FIGURE 7.17 : Structure du playbook `controle_sante.yml`



controle_sante.yml — tableau de santé des 3 serveurs (OPS01, 23/10/2026 09:37) Rendu de la sortie réelle — texte intégral : P-80b

```
# 09:37:03 — $ ansible-playbook playbooks/controle_sante.yml -e sante_rapport=true ; echo "code retour : $?"
TASK [Résultat du contrôle de santé] ****************************************
linux01   Identité                  OK              linux01 192.168.50.20, Ubuntu 24.04, noyau 6.8.0-147-generic, démarré depuis 0.6 h
linux01   Heure                     OK              2026-10-23 09:37:04 CEST, NTP synchronisé : yes
linux01   Espace disque             OK              / 29.2 %  /data 5.3 %  /boot 11.4 %  (seuil 85 %)
linux01   Disque de données /data   OK              /data monté
linux01   Services attendus         OK              ssh, nginx, prometheus-node-exporter actifs
linux01   Unités systemd en échec   OK              aucune
linux01   Journal (erreurs 24 h)    ATTENTION       4 ligne(s) ; dernière : 2026-10-23T09:29:12+02:00 linux01 systemd[1]: Failed to start infraready-controle-site.service - …
linux01   DNS du domaine            OK              win-dc01.infraready.test -> 192.168.50.10
linux01   Site web (HTTP + contenu) OK              http://linux01.infraready.test/ -> HTTP 200
ops01     Identité                  OK              ops01 192.168.50.30, Ubuntu 24.04, noyau 6.8.0-147-generic, démarré depuis 0.6 h
ops01     Heure                     OK              2026-10-23 09:37:04 CEST, NTP synchronisé : yes
ops01     Espace disque             OK              / 26.3 %  /boot 11.4 %  (seuil 85 %)
ops01     Disque de données /data   NON APPLICABLE  pas de disque de données sur cet hôte
ops01     Services attendus         OK              prometheus, grafana-server, prometheus-node-exporter actifs
ops01     Unités systemd en échec   OK              aucune
ops01     Journal (erreurs 24 h)    ATTENTION       2 ligne(s) ; dernière : 2026-10-23T09:03:23+02:00 ops01 sudo[1795]: infraadmin : a password is required ; PWD=/home/infra…
ops01     DNS du domaine            OK              win-dc01.infraready.test -> 192.168.50.10
ops01     Prometheus - cibles       OK              4/4 cibles up
ops01     Grafana - API             OK              base ok, version 13.2.4
win-dc01  Identité                  OK              WIN-DC01 192.168.50.10, Microsoft Windows Server 2025 Standard Evaluation (build 26100), démarré depuis 0.6 h
win-dc01  Heure                     OK              2026-10-23T09:37:12+02:00, source NTP : 0.fr.pool.ntp.org,0x8
win-dc01  Espace disque             OK              C: 37.3 %  (seuil 85 %)
win-dc01  Services attendus         OK              NTDS, DNS, Netlogon, Kdc, W32Time, WinRM, windows_exporter actifs
win-dc01  Journal (erreurs 24 h)    ATTENTION       2 événement(s) System ; 2026-10-23 09:01 41 Microsoft-Windows-Kernel-Power | 2026-10-23 09:01 6008 EventLog
win-dc01  DNS du domaine            OK              linux01.infraready.test -> 192.168.50.20 (serveur DNS local)
win-dc01  Active Directory          OK              domaine infraready.test
win-dc01  windows_exporter          OK              http://localhost:9182/metrics -> HTTP 200
TASK [Aucun contrôle en échec et aucun hôte inaccessible] ********************
    msg: 23 OK, 3 ATTENTION, 1 NON APPLICABLE, 0 ECHEC, 0 INACCESSIBLE
```

FIGURE 7.18 : Contrôle de santé des trois serveurs : 23 OK, 3 ATTENTION, 1 NON APPLICABLE, 0 ECHEC (rendu de la sortie, **P-80b** / **P-82**)

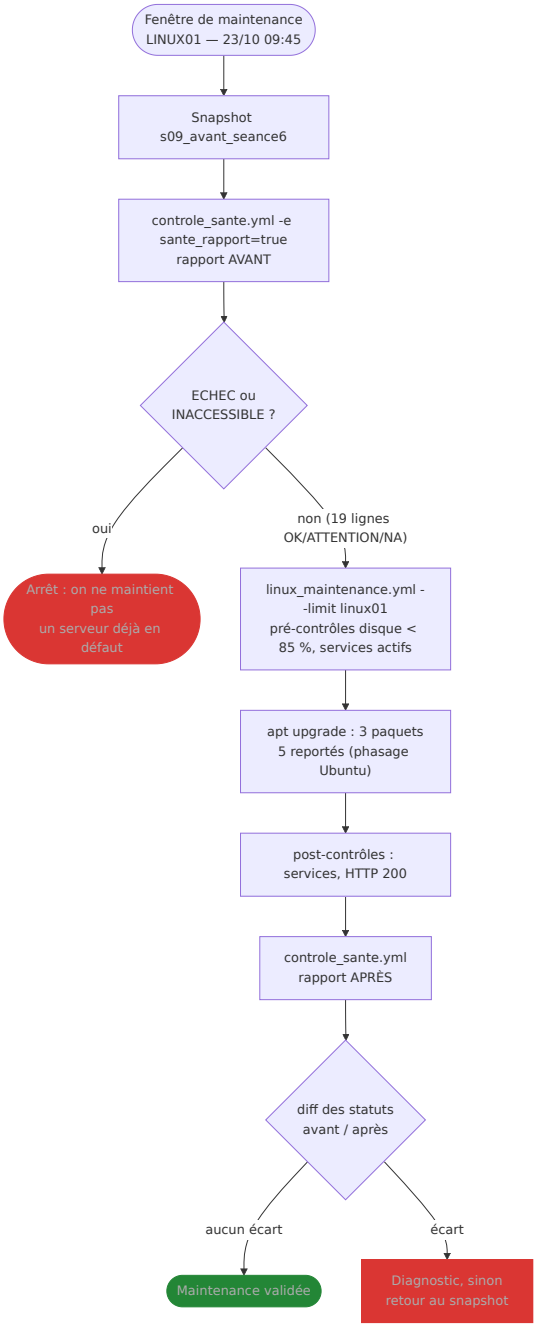


FIGURE 7.19 : Processus de maintenance suivi, avec ses points d'arrêt

Maintenance LINUX01 encadrée par deux contrôles de santé (09:45) Rendu de la sortie réelle — texte intégral : P-87

```
# AVANT — 2026-10-23 09:45:02 CEST — $ ansible-playbook playbooks/controle_sante.yml --limit linux01,ops01,localhost -e sante_rapport=true
# MAINTENANCE — 09:45:12 — $ ansible-playbook playbooks/linux_maintenance.yml --limit linux01
The following upgrades have been deferred due to phasing:
  cloud-init gir1.2-glib-2.0 libglib2.0-0t64 libglib2.0-bin libglib2.0-data
The following packages will be upgraded:
  tmux ubuntu-pro-client ubuntu-pro-client-l10n
3 upgraded, 0 newly installed, 0 to remove and 5 not upgraded.
linux01                    : ok=16   changed=2    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
# APRÈS — 09:45:29 — $ ansible-playbook playbooks/controle_sante.yml --limit linux01,ops01,localhost -e sante_rapport=true
$ diff <rapport de santé avant> <rapport de santé après>  (colonnes hôte / contrôle / statut)
aucun écart de statut entre avant et après
```

FIGURE 7.20 : Maintenance de LINUX01 : 3 paquets installés, 5 reportés par le phasage d'Ubuntu, aucun écart de statut avant / après (rendu de la sortie, **P-87**)

7.9 UC-C3 — Observation ponctuelle et suivi dans le temps

La mesure du jour (OPS01 / à 26,3 %) ne dit rien d'une tendance. L'historique de Prometheus ne couvre que 15 heures sur 91, car le laboratoire était éteint le reste du temps (figure 7.21) : OPS01 gagne environ 0,12 Gio sur 15 heures de fonctionnement, LINUX01 reste stable. Une pente calculée par Prometheus sur 45 minutes de données annonçait au contraire LINUX01 à 4,7 % dans 30 jours : **une pente calculée sur si peu de données n'est pas une tendance**. Une alerte prédictive est proposée, à valider après une semaine de fonctionnement continu.

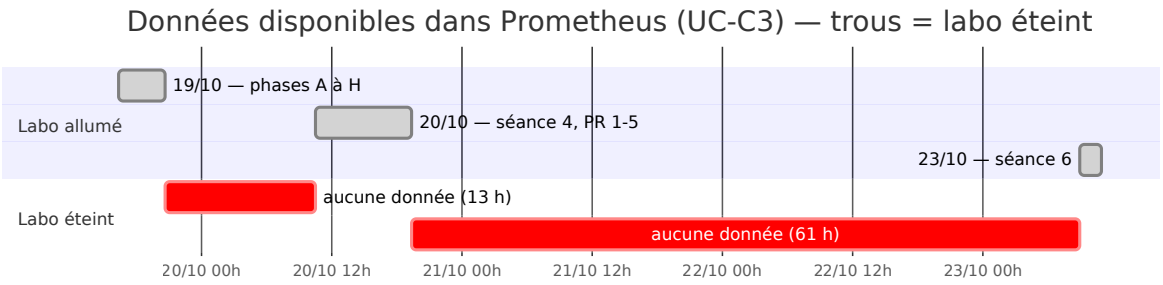


FIGURE 7.21 : Périodes couvertes par l'historique Prometheus



FIGURE 7.22 : Évolution du stockage du 19 au 23/10 : trois îlots de données ; on y voit aussi `C:` de WIN-DC01 passer de 24 % à 43 % puis 37 % pendant Windows Update (**P-88**)

7.10 UC-A3 (défi) — Quotas sur le disque de test

Des quotas ont été préférés à un RAID de démonstration, faute de disques supplémentaires réservés au TP. Le playbook `linux_quotas.yml` vérifie d'abord le numéro de série du disque et ses options de montage, et s'arrête avant toute modification si ce n'est pas le disque de test.



FIGURE 7.23 : Comportement des quotas observé

LINUX01 — test des quotas sur /data (alice bloquée à 250 Mio) Rendu de la sortie réelle — texte intégral : P-89

```
# a) alice écrit 220 Mio : au-dessus du seuil souple (200 Mio) — accepté, délai de grâce de 7 jours
$ sudo -u alice dd if=/dev/zero of=/data/partage-web/test-alice-1.bin bs=1M count=220 status=none; echo "code dd : $?"
code dd : 0
$ quota -s -u alice | tail -1
       /dev/sdb    220M*   200M    250M   7days       1       0       0
# b) alice écrit 100 Mio de plus : bloquée au seuil strict (250 Mio)
$ sudo -u alice dd if=/dev/zero of=/data/partage-web/test-alice-2.bin bs=1M count=100 status=none; echo "code dd : $?"
dd: error writing '/data/partage-web/test-alice-2.bin': Disk quota exceeded
code dd : 1
$ ls -l --block-size=M /data/partage-web/; quota -s -u alice | tail -1
total 250M
-rw-rw-r-- 1 alice webadmins 220M Oct 23 09:52 test-alice-1.bin
-rw-rw-r-- 1 alice webadmins  30M Oct 23 09:52 test-alice-2.bin
       /dev/sdb    250M*   200M    250M   7days       2       0       0
# c) bruno, dans le même dossier, n'est pas affecté par la consommation d'alice
$ sudo -u bruno dd if=/dev/zero of=/data/partage-web/test-bruno.bin bs=1M count=50 status=none; echo "code dd : $?"; quota -s -u bruno | tail -1
code dd : 0
       /dev/sdb  51200K    200M    250M              1       0       0
```

FIGURE 7.24 : Test réel : alice acceptée à 220 Mio avec délai de grâce, puis bloquée à 250 Mio (rendu de la sortie, **P-89**)

Limites : root n'est pas soumis aux quotas ; `repquota` lit un fichier mis à jour en différé, alors que `quota -u` interroge le noyau. Le premier essai n'était pas idempotent (`repquota` sans `-v` masque les comptes sans occupation) : corrigé (**P-89a** → **P-89**).

Chapitre 8

Sécurité, limites d'intervention et recette

8.1 Mesures de sécurité appliquées

Table 8.1 : Mesures de sécurité et de maîtrise des risques

Principe	Mise en œuvre
Périmètre limité	Seules les VM 100 à 102 ont été modifiées ; les autres machines de l'hôte n'ont pas été touchées
Isolation réseau	Laboratoire sans accès au réseau domestique (nftables), accès Internet par traduction d'adresses
Retour arrière	Snapshots avant chaque changement important : `s01` à `s09` (installation, AD DS, mises à jour, supervision, Ansible, maintenance, séances 4 et 6)
Cible vérifiée avant toute opération destructive	Disque de test identifié par son numéro de série, cinq contrôles avant formatage, garde-fou dans `linux_quotas.yml`
Moindre privilège	Comptes Linux verrouillés, droits par groupe, support informatique en lecture seule, compte de service `ansible` dédié, service du contrôle planifié sans capacité
Ports au strict besoin	UFW en refus par défaut sur les VM Linux, WinRM et ICMP limités au laboratoire, exporters ouverts à OPS01 seulement
Secrets	Ansible Vault AES256 ; modèles d'installation à variables ; fichiers de réponse supprimés après usage ; recherche des secrets avant chaque commit ; gitleaks et contrôle du chiffrement des vaults en CI
Changements perturbateurs documentés	Redémarrage de promotion du DC, réseau de l'hôte (sauvegarde), DNS des VM Linux (sauvegarde netplan)
Validation des changements sensibles	La correction de l'incident du DC n'est pas appliquée sans accord explicite de l'équipe

8.2 Limites connues

— WinRM en HTTP (chiffrement NTLM du message, sans certificat TLS) ; Prometheus et Grafana sans TLS (laboratoire isolé) ; Grafana consultable sans compte en lecture seule depuis le laboratoire.

— Pas d'Alertmanager : les alertes sont visibles dans Prometheus et Grafana mais ne déclenchent aucune notification.

— La supervision est hébergée sur le même hôte que les serveurs supervisés : elle ne voit pas les arrêts de l'hôte (incident UC-05).

— Mot de passe sudo de l'hôte Proxmox faible, à changer ; jeton API Proxmox utilisé pendant la séance 6 à révoquer.

— La mise à jour KB5007651 est exclue de l'installation automatique et doit être suivie à la main.

8.3 Recette finale

TABLE 8.2 : Recette finale par phase

Phase	Validation finale	État	Preuves
A	VM, adresses, réseau, isolation, DNS	**Fait**	P-01 à P-05
B	AD DS, groupes, comptes, DNS, journaux	**Fait**	P-10 à P-15
C	Permissions, service, stockage, journaux	**Fait**	P-20 à P-26
D	Playbooks Linux et Windows, idempotence	**Fait**	P-30 à P-39
E	Mises à jour, maintenance	**Fait**	P-40 à P-43
F	Exporters, dashboard, alerte	**Fait**	P-50 à P-56
G	Pipeline GitHub Actions vert	**Fait** (contributions à compléter)	P-60 à P-64
H	Incident corrigé, retour à la normale	**Fait**	P-70 à P-73
Séance 4	Scripts, processus, 6 scénarios de panne, mode check	**Fait** (bilans à rédiger)	P-14, P-25, P-26, P-38, P-39, P-74 à P-79
Séance 6	État initial, collecte, dashboard, alertes, contrôle de santé, tâche planifiée, maintenance, tendance, quotas	**Fait** ; UC-05 **Partiel**	P-80 à P-89, P-64

8.4 Checklist de fin de séance 6 (support, section 6.1)

Table 8.3 : Checklist de fin de séance 6

Exigence	État
État initial documenté pour les systèmes du périmètre	**Fait**
Collecte des métriques Linux et Windows vérifiée	**Fait**
Tableau de bord avec indicateurs lisibles et unités explicites	**Fait**
Au moins une alerte définie et testée	**Fait** (2)
Incident analysé avec journaux, hypothèse, correction et test de retour au service	**Partiel**
Résultats, configurations et documentation intégrés au dépôt	**Fait**
Pipeline CI vérifié, sans prétendre qu'il déploie dans le réseau privé	**Fait**
Aucun secret publié ; captures et journaux expurgés	**Fait**
Compléments et défi avancé	**Fait**

Chapitre 9

Bilan

9.1 Principales difficultés

Le projet a rencontré une quarantaine de difficultés, toutes documentées avec leur cause et leur correction dans les chapitres précédents. Elles se regroupent en quatre familles :

Table 9.1 : Familles de difficultés et enseignements

Famille	Exemples	Enseignement
Environnement	ISO Ubuntu erronée, carte réseau e1000, absence d'iptables, VM éteintes après un redémarrage de l'hôte	Vérifier la plateforme réelle avant de dérouler la procédure prévue
Idempotence	PTR du DC réécrit à chaque passage, DNS `::1`, quotas réécrits, handler non exécuté	Un second passage est un test, pas une formalité : il révèle des conflits entre l'automatisation et le système
Vérifications trompeuses	Assertion réussie sur 0 cible, HTTP 200 avec un mauvais contenu, `--check` qui n'exécute pas les lectures, arrêt « TASK OK » d'une VM tuée	Une vérification doit échouer quand elle ne vérifie rien ; tester aussi le cas négatif
Mises à jour	Boucle Windows Update sur KB5007651, lecteur CD vu comme volume plein, collecteur apt imprécis	Croiser plusieurs sources (historique, journaux, état réel) avant de conclure

9.2 Écarts au sujet

— **Proxmox VE 9.1 au lieu de VMware Workstation** — à faire valider par le formateur (avantages : snapshots, ajout de disque à chaud, installations et snapshots pilotables par API).

— Ressources légèrement supérieures au plan (vCPU) ; disque Windows en IDE (choix initial conservé).

— Ubuntu 24.04.**5**, dernière révision de la version LTS demandée.

9.3 Fonctionnalités inachevées

Table 9.2 : Points ouverts et action nécessaire

Point	Action nécessaire	Statut
Arrêt non propre de WIN-DC01 (INC-2026-10-23-01)	Validation de l'équipe, application de `Shutdown WithoutLogon = 1`, test d'arrêt réel	Partiel
Bilans individuels (séance 4) et revue croisée	À rédiger par chaque membre	À faire
Contributions de Kenzi Bendjelloul dans l'historique Git	Commits à ajouter par l'intéressé	À faire
Notification des alertes	Ajouter Alertmanager (messagerie de l'équipe)	À faire
KB5007651	Suivi manuel, retrait de l'exclusion quand Microsoft aura corrigé	À faire
Démarrage automatique des VM	Décider de l'option `onboot` sur l'hôte	À faire

9.4 Perspectives

— **Supervision hors de l'hôte** : une sonde externe (ou un second hôte) pour détecter un arrêt complet du laboratoire, angle mort mis en évidence par l'incident UC-05.

— **Chiffrement des flux d'administration** : WinRM en HTTPS, TLS pour Prometheus et Grafana.

— **Centralisation des journaux** (Loki et Grafana Alloy) pour corréler plus vite les événements de plusieurs serveurs.

— **Alerte prédictive sur le disque**, à valider après une semaine de fonctionnement continu.

— **Intégration d'Ubuntu au domaine Active Directory** pour centraliser les identités.

9.5 Bilan de l'équipe

Le projet a atteint ses objectifs : un laboratoire hybride entièrement décrit par un dépôt Git, des configurations rejouables et idempotentes, des secrets chiffrés, une supervision qui détecte à la fois l'indisponibilité d'un serveur et celle d'un service, et une chaîne de livraison où chaque changement est validé automatiquement puis relu. La règle « une preuve par résultat, y compris les échecs » a structuré le travail : la plupart des défauts corrigés ont été trouvés en vérifiant qu'une vérification pouvait échouer.

Bilans individuels

Les bilans individuels demandés par les supports des séances 4 et 6 (opération maîtrisée, commande la plus utile, difficulté isolée, risque identifié, reproductibilité) sont à rédiger par chaque membre de l'équipe ; ils ne sont pas rédigés à leur place dans ce rapport.

Conclusion

InfraReady disposait au départ de serveurs administrés à la main. Elle dispose désormais d'une infrastructure de laboratoire dont l'état est reproductible depuis le dépôt, dont la maintenance s'arrête d'elle-même si les conditions ne sont pas réunies, et dont les anomalies sont visibles avant les utilisateurs. Les écarts et limites sont identifiés et accompagnés d'une action ; le principal point ouvert, l'arrêt non propre du contrôleur de domaine, est compris, reproduit et sa correction est prête. Ce socle peut être étendu sans remise en cause, dans l'ordre des perspectives ci-dessus.

Annexe A

Matrice des preuves

Chaque preuve est un fichier de `docs/preuves/` : sortie horodatée de commandes réelles (`.txt`) ou capture (`.png`). La matrice ci-dessous est générée à partir du dossier technique par `rapport/generer-annexes.py`.

Table A.1 : Matrice des preuves (`docs/preuves/`)

Preuve	Date	Contenu	Fichier(s)
P-01	19/10/2026	VM Proxmox, snapshots, pont vmbr1, règles nftables	`P-01-proxmox-vms-et-reseau.txt`
P-02	19/10/2026	LINUX01 : IP, connectivité, Internet, isolation + contre-test	`P-02-linux01-ip-connectivite-isolation.txt`
P-02b	19/10/2026	OPS01 : IP, connectivité, Internet, isolation	`P-02b-ops01-ip-connectivite-isolation.txt`
P-03	19/10/2026	Capture : WIN-DC01 installé, Server Manager	`P-03-win-dc01-installe-server-manager.png`
P-04	19/10/2026	LINUX01 : DNS du domaine (A, PTR, SRV)	`P-04-linux01-resolution-dns-domaine.txt`
P-04b	19/10/2026	OPS01 : DNS du domaine (A, PTR, SRV)	`P-04b-ops01-resolution-dns-domaine.txt`
P-04c	19/10/2026	WIN-DC01 : IP, ping et résolution des serveurs	`P-04c-win-dc01-ip-dns.txt`
P-05	19/10/2026	Arbre complet des snapshots Proxmox des 3 VM, snapshot final s05	`P-05-proxmox-snapshots-arbre-complet.txt`
P-10a	19/10/2026	1er essai de promotion en échec (`reboot_timeout`)	`P-10a-adds-premier-essai-echec-reboot_timeout.txt`
P-10	19/10/2026	Installation AD DS + forêt infraready.test	`P-10-adds-installation-foret.txt`
P-11	19/10/2026	OU, groupes, comptes AD	`P-11-ad-ou-groupes-comptes.txt`
P-12	19/10/2026	État du domaine, comptes, DNS, zones, dcdiag	`P-12-domaine-dns-verifications.txt`
P-12b	19/10/2026	dcdiag tests fonctionnels + NTP	`P-12b-dcdiag-tests-fonctionnels-ntp.txt`
P-13	19/10/2026	Journaux Windows (Security, Directory Service, DNS, System)	`P-13-journaux-windows-ad.txt`
P-14	20/10/2026	Windows : état initial, services, processus, journal System, script `Check-DnsClient.ps1` (0/1/2)	`P-14-win-dc01-processus-services-script-dnsclient.txt`
P-15	20/10/2026	`windows_adds.yml` idempotent après correction du DNS du DC (`changed=0` ×2)	`P-15-windows-adds-dns-idempotence.txt`

Preuve	Date	Contenu	Fichier(s)
P-20	19/10/2026	Comptes, groupes, permissions Linux + tests de droits	`P-20-linux01-comptes-groupes-permissions.txt`
P-21	19/10/2026	nginx, service et processus	`P-21-linux01-service-nginx-processus.txt`
P-22	19/10/2026	Disque de test : contrôles, formatage, montage UUID	`P-22-linux01-disque-test-montage.txt`
P-23	19/10/2026	Mise à jour contrôlée d'Ubuntu	`P-23-linux01-mise-a-jour-controlee.txt`
P-24	19/10/2026	Analyse des journaux Linux	`P-24-linux01-analyse-journaux.txt`
P-25	20/10/2026	Script `check-nginx.sh` testé dans 3 situations (codes 0/1/2)	`P-25-script-check-nginx-trois-situations.txt`
P-26	20/10/2026	LINUX01 : état initial, processus, mémoire, disque, services, journaux nginx	`P-26-linux01-processus-services-journaux.txt`
P-30	19/10/2026	Ansible : version, inventaire, connexions	`P-30-ansible-inventaire-connexions.txt`
P-31	19/10/2026	Playbook Linux — 1er passage	`P-31-playbook-linux-1er-passage.txt`
P-32	19/10/2026	Playbook Linux — 2e passage (idempotence) + UFW	`P-32-playbook-linux-2e-passage-idempotence.txt`
P-33	19/10/2026	Playbook Windows — 1er passage	`P-33-playbook-windows-1er-passage.txt`
P-34	19/10/2026	Playbook Windows — 2e passage (idempotence)	`P-34-playbook-windows-2e-passage-idempotence.txt`
P-35	19/10/2026	Playbook Windows — correction PTR du DC	`P-35-playbook-windows-correction-ptr-dc.txt`
P-36	19/10/2026	Diagnostic : enregistrements de la zone inverse (statique/dynamique)	`P-36-diagnostic-ptr-dc-statique-vs-dynamique.txt`
P-36b	19/10/2026	Zone inverse après réenregistrement du DC	`P-36b-ptr-dc-apres-reenregistrement-dynamique.txt`
P-37	19/10/2026	Playbook Windows stable après réenregistrement du DC (changed=0 ×2)	`P-37-playbook-windows-ptr-dc-stable.txt`
P-38	20/10/2026	`linux_base.yml` + `site_message` et contrôle de contenu — 1er passage	`P-38-playbook-linux-site-message-1er-passage.txt`
P-38b	20/10/2026	`linux_base.yml` — 2e passage (`changed=0`)	`P-38b-playbook-linux-site-message-2e-passage.txt`
P-39a	20/10/2026	Simulation `--check --diff` en échec (blkid / uri non exécutés)	`P-39a-playbook-linux-mode-check-echec.txt`
P-39	20/10/2026	Simulation corrigée (`check_mode: false`) puis exécution `changed=0`	`P-39-playbook-linux-mode-check-corrige.txt`
P-40	19/10/2026	Maintenance Linux (linux01, ops01) : pré-contrôles, apt, post-contrôles	`P-40-linux-maintenance-standard.txt`
P-40b	19/10/2026	Rapports de maintenance générés (`~/infraready-logs/`)	`P-40b-rapports-maintenance-generes.txt`
P-41a	19/10/2026	Maintenance Windows — 1er essai : lecteur CD vu comme volume plein	`P-41a-windows-maintenance-1er-essai-lecteur-cd.txt`
P-41	19/10/2026	Maintenance Windows — recherche des mises à jour (5 trouvées)	`P-41-windows-maintenance-recherche-maj.txt`

Preuve	Date	Contenu	Fichier(s)
P-42	19/10/2026	Maintenance Windows — installation (5 MAJ, redémarrage du DC, boucle KB5007651)	`P-42-windows-maintenance-installation-maj.txt`
P-42b	19/10/2026	Diagnostic WIN-DC01 : historique WU, journal Système, plateforme Defender	`P-42b-win-dc01-diagnostic-kb5007651-historique-wu.txt`
P-42c	19/10/2026	Maintenance Windows — passage complet après exclusion de KB5007651	`P-42c-windows-maintenance-installation-complete-apres-exclusion.txt`
P-43	19/10/2026	unattended-upgrades : configuration, minuteries, simulation	`P-43-linux01-mises-a-jour-automatiques-unattended.txt`
P-70	19/10/2026	Incident nginx : reproduction contrôlée et constat du symptôme (404)	`P-70-incident-nginx-reproduction-et-symptome.txt`
P-71	19/10/2026	Incident nginx : diagnostic par élimination, cause confirmée	`P-71-incident-nginx-diagnostic.txt`
P-72a	19/10/2026	Incident nginx : 1er essai de correction, handler non exécuté	`P-72a-incident-nginx-correction-1er-essai-handler-non-execute.txt`
P-72	19/10/2026	Incident nginx : correction avec le playbook corrigé (reproduction n°2)	`P-72-incident-nginx-correction-playbook-corrige.txt`
P-73	19/10/2026	Incident nginx : retour à la normale et 2e passage `changed=0`	`P-73-incident-nginx-retour-normale-et-second-passage.txt`
P-74	20/10/2026	Scénario 1 : variable incorrecte	`P-74-scenario1-variable-incorrecte.txt`
P-75	20/10/2026	Scénario 2 : variable Jinja2 inexistante	`P-75-scenario2-variable-jinja2-inexistante.txt`
P-76	20/10/2026	Scénario 3 : mauvais chemin de destination	`P-76-scenario3-mauvais-chemin-de-destination.txt`
P-77	20/10/2026	Scénario 4 : handler non notifié	`P-77-scenario4-handler-non-notifie.txt`
P-78	20/10/2026	Scénario 5 : nginx arrêté	`P-78-scenario5-nginx-arrete.txt`
P-79	20/10/2026	Scénario 6 : contrôle fonctionnel du contenu	`P-79-scenario6-controle-fonctionnel-contenu.txt`
P-80	23/10/2026	Séance 6 : état initial des 3 serveurs avant toute modification, blocages initiaux	`P-80-seance6-etat-initial-trois-serveurs.txt` `P-80-rendu-win-dc01-evenements-arret.png`
P-80c	23/10/2026	Arbre des snapshots des 3 VM (API Proxmox), dont `s09_avant_seance6`	`P-80c-proxmox-snapshots-s09.txt` `P-80c-rendu-proxmox-snapshots-s09.png`
P-80b	23/10/2026	`openipmi` désactivé par Ansible ; contrôle de santé sans ECHEC	`P-80b-seance6-correction-openipmi-et-controle-sante-vert.txt`
P-81a	23/10/2026	Redéploiement de la supervision : écart de règles corrigé, vérification des cibles instable	`P-81a-seance6-redeploiement-monitoring-verification-cibles-instable.txt`
P-81b	23/10/2026	Vérification des cibles corrigée (attente du 1er scrape), idempotence	`P-81b-seance6-monitoring-verification-cibles-corrigee.txt`
P-81	23/10/2026	Cibles attendues / observées, endpoints, métriques Linux et Windows, règles	`P-81-seance6-collecte-metriques-linux-windows.txt` `P-81-prometheus-cibles-up.png`

Preuve	Date	Contenu	Fichier(s)
P-82	23/10/2026	`controle_sante.yml` : exécution, `--check`, hôte injoignable, correctif `--limit`	`P-82-controle-sante-ansible-lecture-seule.txt` `P-82-rendu-tableau-controle-sante.png`

P-83	23/10/2026	Tâche planifiée : timer systemd, journal, métriques textfile, 2 nouvelles règles	`P-83-controle-planifie-timer-systemd.txt`
			`P-83-rendu-timer-journal-metriques.png`
			`P-83-prometheus-regles-chargees.png`

P-84	23/10/2026	Alertes `SiteWebIndisponible` et `ControlePlanifieEnRetard` : déclenchement et retour à la normale + capture	`P-84-alerte-sitewebindisponible-test-controle.txt`
			`P-84-prometheus-alerte-sitewebindisponible-firing.png`
			`P-84-grafana-dashboard-site-en-echec.png`
			`P-84-grafana-dashboard-controle-en-retard.png`
			`P-84-prometheus-historique-controle-succes.png`
			`P-84-prometheus-historique-alertes.png`

P-85	23/10/2026	Dashboard Grafana v3 : déploiement, vérification par l'API + capture	`P-85-grafana-dashboard-v3-deploiement.txt` `P-85-grafana-dashboard-v3-exploitation.png`
P-86	23/10/2026	Incident INC-2026-10-23-01 : arrêt non propre de WIN-DC01, diagnostic et reproduction	`P-86-incident-arret-non-propre-win-dc01-diagnostic.txt` `P-86-rendu-proxmox-arrets-vm100-reproduction.png`
P-87	23/10/2026	Maintenance LINUX01 avec contrôle de santé avant / après	`P-87-maintenance-linux01-controle-avant-apres.txt` `P-87-rendu-maintenance-avant-apres.png`
P-88	23/10/2026	Espace disque : observation ponctuelle et historique Prometheus	`P-88-observation-ponctuelle-vs-historique-disque.txt` `P-88-grafana-evolution-stockage-19-23-oct.png`
P-89a	23/10/2026	Quotas — 1er essai non idempotent (`repquota` sans `-v`)	`P-89a-quotas-1er-essai-non-idempotent.txt`
P-89	23/10/2026	Quotas sur `/data` : idempotence, dépassement souple / strict, nettoyage	`P-89-quotas-disque-test-data-linux01.txt` `P-89-rendu-test-quotas-alice.png`
P-50a	19/10/2026	Monitoring — 1er essai : échec de connexion admin Grafana	`P-50a-ops01-grafana-1er-essai-echec-mot-de-passe-ini.txt`
P-50	19/10/2026	Playbook Prometheus + Grafana (cibles up, admin OK)	`P-50-ops01-prometheus-grafana-installation.txt`
P-50b	19/10/2026	Playbook monitoring — 2e passage (changed=0)	`P-50b-ops01-monitoring-2e-passage-idempotence.txt`
P-51	19/10/2026	Capture : cibles Prometheus (4/4 up)	`P-51-prometheus-targets.png`
P-52	19/10/2026	Cibles, règles, métriques réelles, exporters, ports	`P-52-prometheus-cibles-metriques-regles.txt`

Preuve	Date	Contenu	Fichier(s)
P-53	19/10/2026	Capture : dashboard Grafana, état normal	`P-53-grafana-dashboard-infraready.png`
P-54	19/10/2026	Démonstration InstanceDown : chronologie + capture Prometheus	`P-54-alerte-instancedown-demonstration.txt` `P-54-prometheus-alerte-instancedown.png`
P-55	19/10/2026	Capture : dashboard pendant l'alerte (LINUX01 hors ligne)	`P-55-grafana-dashboard-alerte-linux01-hors-ligne.png`
P-56	20/10/2026	Dashboard Grafana v2 (variable Hôte, Uptime) déployé, idempotence, requêtes vérifiées	`P-56-grafana-dashboard-v2-deploiement-idempotence.txt`
P-60a	19/10/2026	GitHub Actions sur le dépôt de travail initial : 1er run en échec, 2e vert	`P-60a-github-actions-depot-de-travail-initial.txt`
P-60	20/10/2026	GitHub Actions sur `jolanallen/Infra Ready` : run vert, jobs, déclencheurs, auteurs	`P-60-github-actions-pipeline-vert.txt`
P-61	20/10/2026	Validation des PR #2 et #4 : `verif-idempotence.sh` sur `windows_base` et `linux_base` (IDEMPOTENT)	`P-61-validation-pr-idempotence-playbooks.txt`
P-62	20/10/2026	Tests négatifs (non idempotent → 1, échec → 2, vault absent ou invalide) ; diagnostic du DNS de `windows_adds`	`P-62-validation-pr-tests-negatifs.txt`
P-63	20/10/2026	Validation de la PR #1 : calcul d'occupation disque de `linux_maintenance` (simulation)	`P-63-validation-pr1-maintenance-linux-calcul-disque.txt`
P-64	23/10/2026	GitHub Actions sur les PR de la séance 6 : contrôles exécutés sur chaque branche ; PR #8 à #14 fusionnées	`P-64-github-actions-pr-seance6.txt` `P-64-github-pr-seance6-fusionnees.png` `P-64-github-actions-runs-verts.png`

Annexe B

Arborescence du dépôt

```
InfraReady/
├── .github/workflows/validation.yml     pipeline de validation (4 jobs)
├── ansible/
│   ├── ansible.cfg · requirements.yml
│   ├── inventory/hosts.yml
│   ├── inventory/group_vars/            all, linux, windows, monitoring (vaults chiffrés)
│   └── playbooks/                       windows_adds, windows_base, windows_maintenance,
│       │                                linux_base, linux_maintenance, linux_quotas,
│       │                                ops01_monitoring, controle_sante
│       └── templates/                   index.html.j2, nginx-infraready.conf.j2
├── lab/                                 modèles autoinstall / autounattend, réseau Proxmox
├── monitoring/
│   ├── prometheus/                      prometheus.yml, rules/infraready.rules.yml
│   └── grafana/                         dashboard infraready.json, provisioning
├── scripts/                             amorçage, vérification, diagnostic (Bash, PowerShell)
├── docs/
│   ├── procedure-maintenance.md · rapport-incident.md
│   └── preuves/                         P-01 … P-89 (texte horodaté et captures)
├── rapport/                             sources LaTeX de ce rapport, schémas, script de build
├── InfraReady - Dossier technique.md    dossier technique (Obsidian)
├── InfraReady - Séance 6.md             note de synthèse illustrée de la séance 6
├── InfraReady-Rapport-final.tex / .pdf  ce rapport
├── CLAUDE.md                            règles de contribution (Git, documentation, validation)
└── README.md
```

Annexe C

Journal des changements

Changements appliqués au laboratoire, dans l'ordre chronologique, avec leur validation et le moyen de retour arrière (`sNN_…` = snapshot Proxmox).

Table C.1 : Journal des changements sur le laboratoire

Date	Modification	Validation	Preuve / retour arrière
19/10/2026 14:42	Pont `vmbr1` + NAT/isolation nftables sur l'hôte	`ifreload -a`, ping/isolation OK	P-01, P-02 / `interfaces.bak-infraready`
19/10/2026 14:47	Installations automatiques des 3 VM	3 VM joignables	P-01, P-03 / —
19/10/2026 14:58	Snapshot de base	`qm listsnapshot`	`s01_installation_ok`
19/10/2026 15:05	AD DS + forêt infraready.test	dcdiag fonctionnel OK	P-10, P-12b / `s02_avant_adds`
19/10/2026 15:14	OU, groupes, comptes AD	Membres conformes	P-11 / `s02_avant_adds`
19/10/2026 15:20	NTP externe sur le DC	Stratum 2	P-12b / —
19/10/2026 15:23	Comptes Linux, nginx, disque `/data`	Tests de droits OK	P-20 à P-22 / `s02_avant_maj_c`
19/10/2026 15:24	`apt upgrade` LINUX01 (20 paquets)	Code retour 0	P-23 / `s02_avant_maj_c`
19/10/2026 15:28	Playbooks Linux et Windows (2 passages)	changed=0 au 2e passage	P-31 à P-34 / `s03_avant_ansible`
19/10/2026	DNS des VM Linux → DC, PTR du DC	A/PTR/SRV résolus	P-04, P-35 / `/root/50-cloud-init.yaml.bak-dns`
19/10/2026 16:30	Prometheus, exporters, Grafana sur OPS01	Cibles UP, dashboard, alerte testée	P-50 à P-55 / `s04_avant_maintenance_e`
19/10/2026 16:48	Maintenance Linux (LINUX01, OPS01), `unattended-upgrades`	Post-contrôles OK, reboot non requis	P-40, P-43 / `s04_avant_maintenance_e`

Date	Modification	Validation	Preuve / retour arrière
19/10/2026 16:54	Windows Update sur WIN-DC01 (5 MAJ, redémarrage 18:04)	`failed=1` (boucle KB5007651) puis passage complet OK	P-42 à P-42c / `s04_avant_maj_windows`
19/10/2026 19:21	`reject_list: [KB5007651]` + gabarit du rapport Windows	`failed=0`, rapport lisible	P-42c / —
19/10/2026 19:27	Incident contrôlé : `root` nginx modifié sur LINUX01	404 constaté depuis OPS01	P-70 / `/root/infraready.nginx.bak-incident-h`
19/10/2026 19:30	`flush_handlers` avant les vérifications dans `linux_base.yml` ; rejeu sur LINUX01	HTTP 200, `changed=0` au 2e passage	P-72, P-73 / —
19/10/2026 19:37	Snapshot final des 3 VM	`qm listsnapshot`	P-05 / `s05_phases_a_h_terminees`
20/10/2026 12:50	Snapshot des 3 VM avant la séance 4	`qm snapshot`	`s07_avant_seance4`
20/10/2026 12:51	`site_message` + contrôle HTTP et contenu dans `linux_base.yml` (2 passages)	`changed=1` puis `changed=0`	P-38, P-38b / `s07_avant_seance4`
20/10/2026 12:52	Scripts `check-nginx.sh` et `Check-DnsClient.ps1` (3 situations chacun)	Codes 0 / 1 / 2	P-25, P-14 / —
20/10/2026 12:57	`check_mode: false` sur 4 tâches en lecture seule de `linux_base.yml`	Simulation `changed=0 failed=0`	P-39a, P-39 / —
20/10/2026 12:58	6 scénarios de panne contrôlée (7.4), un à la fois, jusqu'à 13:08	Retour à l'état initial, `changed=0`	P-74 à P-79 / `.etat-initial`, `s07_avant_seance4`
20/10/2026 14:15	Snapshot des 3 VM avant validation des PR #1 à #5	`qm snapshot`	`s08_avant_validation_pr3_5`
20/10/2026 14:16	Dashboard Grafana v2 déployé (`ops01_monitoring.yml`, 2 passages)	IDEMPOTENT, version 2 chargée	P-56 / `s08_avant_validation_pr3_5`
20/10/2026 14:18	`windows_base.yml` et `linux_base.yml` via `verif-idempotence.sh`	IDEMPOTENT ×2	P-61 / —
20/10/2026 14:21	Tests négatifs ; exécution réelle non voulue de `windows_adds.yml` (DNS du DC rejoué, valeur inchangée)	Résolution DNS OK	P-62 / `s08_avant_validation_pr3_5`
20/10/2026 14:24	`linux_maintenance.yml` en simulation (PR #1)	Calcul identique à P-40	P-63 / —
20/10/2026 14:26	Correctif DNS de `windows_adds.yml` testé (PR #6), puis copie d'OPS01 remise à la version de `main`	IDEMPOTENT	P-15 (PR #6) / `s08_avant_validation_pr3_5`
23/10/2026 09:05	Snapshot des 3 VM avant la séance 6 (API Proxmox)	Tâches `qmsnapshot` OK	`s09_avant_seance6`
23/10/2026 09:06	État initial des 3 serveurs (lecture seule)	5 écarts relevés	P-80 / —

Date	Modification	Validation	Preuve / retour arrière
23/10/2026 09:17	Copie du projet sur OPS01 resynchronisée ; `ops01_monitoring.yml` rejoué (5 règles) puis corrigé (attente du 1er scrape)	IDEMPOTENT	P-81a, P-81b / `s09_avant_seance6`
23/10/2026 09:22	Timer `infraready-controle-site` (`linux_base.yml`) et 2 règles d'alerte (`ops01_monitoring.yml`)	`changed=0` au 2e passage, 7 règles	P-83 / `s09_avant_seance6`
23/10/2026 09:26	Test d'alerte : nginx arrêté sur LINUX01 puis redémarré (09:30)	firing à +3 min 08 s, résolue 27 s après	P-84 / —
23/10/2026 09:35	`openipmi` désactivé (`linux_base.yml`, `ops01_monitoring.yml`)	Contrôle de santé sans ECHEC	P-80b / `s09_avant_seance6`
23/10/2026 09:38	Reproduction : arrêt ACPI de WIN-DC01 demandé sans arrêt forcé	Ignoré pendant 90 s, VM restée allumée	P-86 / —
23/10/2026 09:43	Dashboard Grafana v3 (`ops01_monitoring.yml`)	IDEMPOTENT, version 3 chargée	P-85 / `s09_avant_seance6`
23/10/2026 09:45	Maintenance LINUX01 (3 paquets) entre deux contrôles de santé	Aucun écart de statut	P-87 / `s09_avant_seance6`
23/10/2026 09:48	`/data` remonté avec `usrquota,grpquota` ; quotas alice / bruno (`linux_quotas.yml`)	IDEMPOTENT après correction ; dépassement testé	P-89a, P-89 / `s09_avant_seance6`
23/10/2026 09:54	Test d'alerte : timer du contrôle planifié arrêté (09:54) puis redémarré (10:04)	`ControlePlanifieEnRetard` firing à 10:04:01, résolue à 10:04:17	P-84 / —

Annexe D

Glossaire

AD DS
: Active Directory Domain Services : annuaire Windows (comptes, groupes, ordinateurs, authentification Kerberos).

ACPI Interface standard de gestion de l'alimentation ; l'hyperviseur l'utilise pour demander l'arrêt d'une VM.

Ansible Vault
: Chiffrement AES256 de fichiers de variables Ansible contenant des secrets.

CI Intégration continue : contrôles automatiques exécutés à chaque modification du dépôt (ici GitHub Actions).

DC Contrôleur de domaine (WIN-DC01).

Exporter
: Programme qui expose les métriques d'un système au format Prometheus (node_exporter, windows_exporter).

Gestionnaire (handler)
: Tâche Ansible déclenchée seulement si une autre tâche a changé quelque chose (ex. recharger nginx).

Idempotence
: Propriété d'une automatisation dont un second passage ne modifie plus rien (`changed=0`).

nftables
: Pare-feu et traduction d'adresses du noyau Linux, successeur d'iptables.

Preuve P-xx
: Fichier horodaté de `docs/preuves/` qui justifie un résultat de ce rapport.

PromQL
: Langage de requête de Prometheus.

PTR Enregistrement DNS inverse (adresse IP vers nom).

Scrape
: Collecte périodique des métriques d'une cible par Prometheus (toutes les 15 s ici).

Snapshot
: Instantané d'une VM permettant de revenir à un état antérieur.

Textfile (collecteur)
: Fonction de node_exporter qui publie des métriques écrites dans des fichiers `.prom`.

Timer systemd
: Planification d'un service par systemd, équivalent moderne de cron.

UFW Pare-feu simplifié d'Ubuntu.

WinRM
: Protocole d'administration à distance de Windows, utilisé par Ansible.

Annexe E

Références

— Sujet du projet fil rouge InfraReady et supports pédagogiques des séances 4 (*Stockage et processus*) et 6 (*Maintenance opérationnelle et supervision*), Bachelor 3 — Administration systèmes Linux / Windows.

— Dossier technique du projet : `InfraReady - Dossier technique.md` ; procédure de maintenance : `docs/procedure-maintenance.md` ; rapports d'incident : `docs/rapport-incident.md`.

— Documentations officielles : Proxmox VE, Ansible (collections `ansible.windows`, `microsoft.ad`, `community.general`), Prometheus, Grafana, node_exporter, windows_exporter, Microsoft Learn (AD DS, DNS, Windows Update), Ubuntu Server (autoinstall, unattended-upgrades, quota).

— Dépôt de remise : `https://github.com/jolanallen/InfraReady`.